

SISTEM DE SORTARE OBIECTE ÎN FUNCȚIE DE CULOARE

Lucrare de licență

Candidat: Adela-Maria, Biciușcă

Coordonator științific: Conf. Dr. Ing. Răzvan BOGDAN

Sesiunea: Iunie 2023

Cuprins

1. INTRODUCERE.....	5
1.1. Context.....	5
1.2. Motivație.....	6
2. STADIUL ACTUAL ÎN DOMENIU.....	9
2.1. Aplicații similare.....	9
2.2. Literatura de specialitate.....	10
3. TEHNOLOGII FOLOSITE.....	14
3.1. Raspberry Pi.....	14
3.2. Python - OpenCV.....	17
3.3. Android Studio.....	18
4. ARHITECTURA PROIECTULUI.....	20
4.1. Arhitectura sistemului.....	20
4.2. Arhitectura hardware.....	21
4.2.1. Modul Cameră Raspberry Pi.....	22
4.2.2. Servomotor SG90.....	26
4.3. Arhitectura software.....	27
5. IMPLEMENTARE.....	31
5.1. Algoritm recunoaștere culoare obiecte.....	31
5.2. Programarea servomotor.....	37
5.3. Dezvoltarea aplicației Android Studio.....	40
5.4. Comunicare aplicație - Raspberry Pi.....	42
6. TESTARE.....	45
7. CONCLUZII ȘI PERSPECTIVE.....	47
8. REFERINȚE BIBLIOGRAFICE.....	48
ANEXA 1: Ghidul de utilizare al sistemului.....	50

Lista figurilor

- Figura 1 - Diferență percepere culoare vedere normală și daltonism
- Figura 2 - Pinout Raspberry Pi
- Figura 3 - Arhitectura generală a sistemului
- Figura 4 - Arhitectura hardware a sistemului
- Figura 5 - Portul pentru modulul camerei la Raspberry Pi
- Figura 6 - Aranjarea pinilor pe camera Raspberry Pi
- Figura 7 - Reprezentarea HSV
- Figura 8 - Conexiune Raspberry Pi - servomotor
- Figura 9 - Diagrama UseCase
- Figura 10 - Diagrama UML
- Figura 11 - Exemplu mască pentru recunoașterea culorilor
- Figura 12 - Schemă algoritm detectare culoare
- Figura 13 - Schemă funcționare timp de așteptare
- Figura 14 - Schemă logică funcționare servomotor
- Figura 15 - Rezultate test 3
- Figura 16 - Ecran aplicație Android
- Figura 17 - Utilizare sistem fizic

Lista tabelelor

- Tabelul 1 - Caracteristicile de bază ale aplicațiilor “CSort”, “Color Blind Pal” și “CL colour sorting system”.
- Tabelul 2 - Aranjarea pinilor pe Raspberry Pi
- Tabelul 3 - Aranjarea pinilor pentru modulul camerei pe Raspberry Pi
- Tabelul 4 - Rezultate test 1

Lista secvențelor de cod

- Secvența 1 - Importuri captare și procesare video
- Secvența 2 - Inițializare captură video
- Secvența 3 - Interval HSV roșu și verde
- Secvența 4 - Interval HSV galben și albastru
- Secvența 5 - Declararea variabilelor ce indică recunoașterea sau nu a unei culori
- Secvența 6 - Capturarea frame-urilor video, conversie din BGR în HSV
- Secvența 7 - Crearea de mască pentru detecția roșu și verde
- Secvența 8 - Crearea de mască pentru detecția galben și albastru
- Secvența 9 - Condiție receptare culoare (albastru)
- Secvența 10 - Import bibliotecă pentru timp și secvență așteptare 3 secunde
- Secvența 11 - Import bibliotecă și setup GPIO

Secvența 12 - Rotire servomotor la dreapta și revenire la poziția inițială

Secvența 13 - Rotire servomotor la stânga și revenire la poziția inițială

Secvența 14 - Comenzi terminare program

Secvența 15 - XML aplicație Android

Secvența 16 - Metoda onCreate() și conținut

Secvența 17 - Inițializarea variabilelor și datele de conectare prin SSH

Secvența 18 - Metoda prin care se realizează comunicarea SSH

Abrevieri

GND	Ground
GPIO	General Purpose Input/Output
IoT	Internet of Things
PWM	Pulse Width Modulation
UML	Unified Modeling Language
SSH	Secure Shell
IP	Internet Protocol
OpenCV	Open Source Computer Vision
HSV	Hue, Saturation, Value
BGR	Blue, Green, Red
XML	Extensible Markup Language
JSch	Java Secure Channel

1. INTRODUCERE

1.1. Context

Această lucrare reprezintă încercarea de a ajuta persoanele cu deficit de detecție corectă a culorilor obiectelor și ulterior eficientizarea acestui proces prin sortarea obiectelor.

Recunoscând nevoia unui sistem eficient și ușor de utilizat, voi realiza un proiect care integrează tehnologii de ultimă oră cum ar fi componentele hardware folosite și încorporează o aplicație Android în componentele sale. Folosind această combinație de hardware și software consider că pot realiza un sistem ce va putea atinge obiectivul principal, acela de a ajuta persoanele daltoniste. Sistemul va fi unul ușor utilizabil, printr-o interfață grafică oferită de aplicația Android, astfel utilizatorul va avea nevoie să cunoască doar cum să folosească aplicația, nimic mai mult.

Procesul de sortare se află în centrul sistemului, fiind principala funcționalitate și implică clasificarea obiectelor detectate în diferite containere, în funcție de culoarea pe care o au. În final ne dorim să putem sorta în timp real obiecte de culori roșu sau verde și galben sau albastru, obiecte trecute prin filtrul de sortare. Astfel vom obține un sistem complex ce va îndeplini scopul recunoașterii culorilor obiectelor și sortării acestora, sistem ce este dedicat persoanelor daltoniste și care îi va ajuta pe aceștia să aibă acces la realizarea acestei acțiuni simple.

Principală componentă a sistemului este placa de dezvoltare Raspberry Pi care acționează ca un creier ce pe lângă faptul că va primi și procesa informațiile, aceasta le va trimite eficient prin arhitectura sistemului. Prin integrarea perfectă a unei camere de înaltă rezoluție, sistemul folosește algoritmi de detectare a imaginii pentru a diferenția culorile precise ale obiectelor. Aceste date vizuale esențiale sunt apoi transmise servomotorului SG90, o componentă responsabilă pentru gestionarea mecanismului de sortare. Servomotorul își asumă un rol indispensabil în procesul de sortare, direcționând eficient mișcarea obiectelor pe baza informațiilor de culoare primite. Obiectele vor avea ca punct de pornire dintr-un sistem fizic realizat undeva la 20 cm (centimetri) de locația camerei, acest punct inițial este mai înalt, ceea ce va ajuta obiectul să ajungă prin inerție în locul unde se produce sortarea, adică înainte de servomotor. Pe măsură ce camera detectează culoarea obiectului, acesta se va mișca într-un punct exact unde se afla servomotorul care se va roti stânga sau dreapta, în funcție de culoarea detectată, aceste acțiuni au ca scop direcționarea obiectelor în containere specifice culorii acestora, acestea se află în continuarea servomotorului și sunt specifice pentru fiecare culoare. Placa va face legătura dintre culoarea detectată de cameră și servomotor, iar la recunoașterea unei culori specifice servomotorul se va roti, astfel va permite obiectelor să treacă doar prin partea specifică, în funcție de culoare.

Raspberry Pi își îndeplinește rolul de gestionare a tuturor acțiunilor. Camera recunoaște instantaneu culorile obiectelor transmițând prin placă semnal către servomotor, acesta din urmă având, de asemenea, un răspuns instantaneu la semnalul primit, asigurând un proces fără întreruperi și cu o acuratețe ridicată.

La terminarea lucrării, containerele desemnate așteaptă obiectele sortate pentru depozitare. Acestea au un scop temporal de depozitare sau transportare, în funcție de nevoie. Pentru a facilita interacțiunea utilizatorului și pentru a optimiza controlul asupra funcționalităților sistemului, proiectul include dezvoltarea unei aplicații Android numită „CSort”. Această aplicație servește drept interfață intuitivă prin care utilizatorii pot iniția sau opri procesul de sortare conform cerințelor lor, oferindu-le control asupra sistemului. Prin valorificarea puterii tehnologiei mobile, utilizatorii pot interacționa fără efort cu sistemul.

Aplicația dezvoltată se va numi “CSort” și ne vom referi la aceasta în documentație ca fiind tot sistemul de recunoaștere de culoare, sortare cât și aplicația mobilă în sine care ajută utilizatorul să interacționeze cu sistemul, aceasta valorifică folosirea tehnologiei pentru a ne ușura viața și este centrată pe nevoile utilizatorului.

1.2. Motivație

Ideea a pornit în urma unei postări pe care am văzut-o pe rețeaua de socializare LinkedIn în care un utilizator și-a împărtășit experiența de a trăi cu deficiențe de vedere. În această postare s-a subliniat modul în care, în ultimii ani, au fost dezvoltate diverse sisteme pentru a ajuta persoanele cu deficiențe de vedere să își desfășoare activitățile zilnice la fel de eficient ca și colegii lor cu vedere normală.

Am rezonat cu această postare deoarece cred în importanța incluziunii și a creșterii gradului de conștientizare cu privire la provocările cu care se confruntă diferite grupuri de oameni, indiferent de afecțiunile acestora și cred că este una dintre responsabilitățile noastre să le oferim soluții la problemele cu care aceștia se confruntă. Îmi doresc să realizez acest proiect nu doar ca o soluție practică, ci și ca un mijloc de a promova conștientizarea și înțelegerea persoanelor cu deficiențe de vedere.

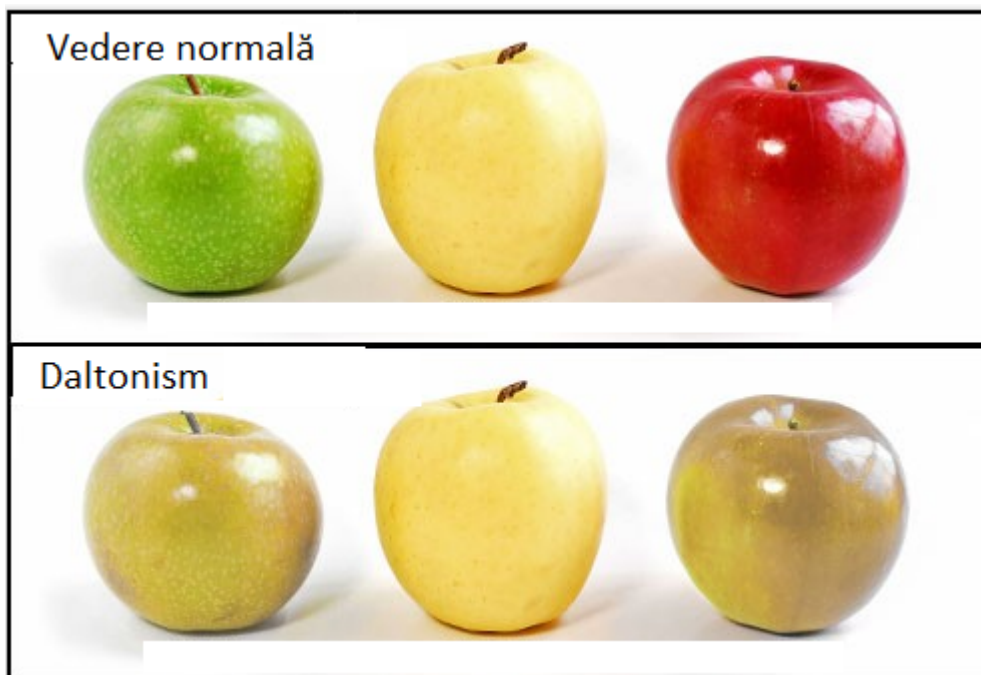
În ultima perioadă se încearcă creșterea conștientizării legate de incluziunea diferitor grupuri de oameni. Mai mult, am observat o dorință comună între oameni de vârstă similară, inclusiv eu, de a atinge un nivel mai mare de independență. Incapacitatea de a vedea corect poate împiedica semnificativ o persoană să se descurce în lume în mod independent.

Un procent de aproximativ 8% dintre bărbați și 3% dintre femei din populația globului [1] suferă de “Colour Vision deficiency” (CVD), cunoscută și sub numele de daltonism, care se referă la o percepție redusă sau incapacitatea de a distinge culorile cu acuratețe. Acest lucru poate produce incapacitatea de a funcționa la fel ca o persoană ce distinge corect culorile, astfel diferite joburi sau chiar activități de timp liber pot fi o povară pentru persoanele ce suferă de “Colour Vision deficiency”, ne referim în română la această afecțiune ca daltonism.

Persoanele ce suferă de daltonism au dificultăți în a distinge anumite culori, cum ar fi roșu și verde sau albastru și galben. Daltonismul este o condiție în care percepția culorilor este adesea limitată și inexactă [2]. Acesta este de obicei ereditară și apare din cauza unor anomalii ale genelor responsabile de vederea culorilor. Pentru persoanele daltoniste unele nuanțe de roșu pot apărea ca maro sau gri, în timp ce nuanțele de verde pot apărea ca bej sau roz. În cazurile severe, persoanele cu daltonism pot vedea lumea în nuanțe de gri. Majoritatea persoanelor cu daltonism pot duce o viață normală, deși pot întâmpina anumite provocări în sarcini care necesită o identificare corectă a culorilor, cum ar fi citirea hărților cu coduri de culoare sau interpretarea semnalelor de culoare.

În Figura 1 se poate diferenția cum percepe culorile o persoană ce are vedere normală și aceleași culori de o persoană daltonistă.

Figura 1 - Diferență percepere culoare vedere normală și daltonism



Prin acest proiect, scopul meu este să fac un pas înainte în a asista persoanele daltoniste în sarcini simple care pot fi provocatoare. Oferindu-le instrumentele pentru a îndeplini aceste sarcini în mod independent, sper să le sporesc sentimentul de apartenență. În plus, îmi propun să sensibilizez un public mai larg despre această afecțiune, promovând înțelegerea și empatia față de persoanele cu deficiențe de recunoaștere a culorilor.

Cred că acest proiect are potențialul de a avea un impact pozitiv asupra vieții persoanelor cu deficiențe de recunoaștere a culorilor. Oferindu-le un mijloc de a depăși limitările impuse de condiția lor. În plus, prin creșterea gradului de conștientizare și

Înțelegere a daltonismului, putem promova o societate mai incluzivă, care îmbrățișează nevoile diverse ale membrilor săi.

2. STADIUL ACTUAL ÎN DOMENIU

2.1. Aplicații similare

Pentru a compara sistemul dezvoltat în cadrul lucrării de diplomă am căutat aplicații asemănătoare cu “CSort”. Acestu lucru a fost o provocare deoarece deși există în literatura de specialitate numeroase proiecte bazate pe studiul detectării de culoare sau obiecte cât și pentru sortarea de obiecte, puține dintre ele sunt aplicații reale ce se pot folosi de către orice utilizator. Majoritatea sistemelor ce îndeplinesc funcție de sortare de obiecte sunt mașini industriale care, bineînțeles, nu sunt ușor accesibile oricărui utilizator.

Am ales din gama de aplicații asemănătoare “Cort” două pentru a compara diferitele funcționalități ale celor deja existente pe piață cu cele ale aplicații dezvoltate de mine.

“Color Blind Pal” [3] este o aplicație ce se găsește pe Google Play. Aceasta are scopul de a asista și ajuta persoanele ce au deficit de distincție a culorii, adică persoanele daltoniste, să înțeleagă ce culoare are un obiect. Ce trebuie să faci este să deschizi aplicația, și folosind camera telefonului, aplicația va afișa pe ecran culoarea obiectului. Un alt scop al aplicației este de a arăta utilizatorilor ce pot distinge corect culorile, cum ar vedea în cazul în care ar suferi de daltonism. Pentru a determina culoarea unui obiect se poate folosi funcția Color Inspector, apoi aplicația va arăta utilizatorului printr-un text culoarea obiectului. O funcționalitate a acesteia ar fi teste de detecție corectă a culorii, prin care poți afla dacă suferi sau nu de daltonism. Comparativ cu “CSort”, aceasta aplicație ajută la detecția culorii însă nu are integrată partea de sortare și deși folosind această aplicație se poate face fizic acțiunea de sortare de către utilizator, scanarea fiecărui obiect separat ar lua timp și energie

“CL colour sorting system” [4] este un sistem ce sortează bazat pe culoare. Are ca rezultat o imagine tipografică color a fiecărui obiect de pe banda transportoare care rulează rapid. După detectare, obiectele individuale sunt clasificate în clase de materiale pe baza unor criterii și caracteristici definite individual. Acest lucru duce apoi la o decizie de sortare. Domeniile tipice de aplicare pentru sortarea culorilor sunt separarea concentratelor de metale grele neferoase în cupru, alamă și metale gri sau sortarea materialelor industriale. Acest sistem are ca scop funcționarea în fabrică, ceea ce ar face imposibilă transportarea sau utilizarea individuală, ceea ce în final nu și-ar atinge unul din obiectivele setate de mine, adică utilizarea în orice moment necesar de către persoanele ce nu recunosc corect culorile.

Cele două sisteme discutate consider că au ca plus recunoașterea unei game mult mai largi de culori, ceea ce pentru persoanele ce suferă de daltonism nu este neapărat necesar. “Color Blind Pal” este un asistent perfect pentru persoanele ce suferă de daltonism, în timp ce “CL colour sorting system” ajută la recunoașterea și sortarea industrială, neavând scopul de a ajuta comunitatea, în schimb are specificații complexe și complete pentru a-și îndeplini propriile obiective.

O analiză comparativă între cele trei sisteme “CSort”, “Color Blind Pal” și “CL colour sorting system” se poate găsi în Tabelul 1. Funcționalitățile analizate regăsite într-o aplicație sunt marcate cu “✓”, iar cele absente cu “✗”.

Tabelul 1 - Caracteristicile de bază ale aplicațiilor “CSort”, “Color Blind Pal” și “CL colour sorting system”.

Caracteristici și funcționalități	CSort	Color Blind Pal	CL colour sorting system
Dezvoltare hardware	✓	✗	✓
Magazin de aplicații	✗	✓	✗
Detectie culoare	✓	✓	✓
Sortare obiecte	✓	✗	✓
Aplicație mobilă	✓	✓	✗
Dedicat utilizator singular	✓	✓	✗
Utilizare în fabrica	✗	✗	✓
Utilizare cameră	✓	✓	✓
Utilizare cameră 3D	✗	✗	✓

2.2. Literatura de specialitate

Pe lângă aplicațiile menționate anterior, disponibile în prezent pe piață, există și alte aplicații dezvoltate recent în cadrul comunității științifice care se încadrează în același domeniu și anume recunoașterea culorilor obiectelor și sortarea corespunzătoare a acestora în funcție de culoare folosind tehnologiile IoT. Aceste studii au contribuit semnificativ la progresul sistemelor de recunoaștere și sortare a culorilor, deschizând calea pentru soluții inovatoare în acest domeniu.

O astfel de lucrare a fost realizată de Francesco Bianconi, Antonio Fernández, Fabrizio Smeraldi și Giulia Pascoletti în “Colour and Texture Descriptors for Visual Recognition: A Historical Overview” [5]. Această publicație a apărut în 19 noiembrie 2021 și oferă o prezentare istorică a descriptorilor de culoare și texturi pentru percepția vizuală. Acest articol evidențiază trecerea de la abordări teoretice (bazate pe crearea manuală a descriptorilor) la soluții bazate pe deep learning, care domină în prezent domeniul recunoașterii vizuale. Prin acest articol se poate înțelege relevanța și importanța acestor

studii pentru îmbunătățirea vizuala și recunoașterea culorilor. Autorii încep prin a explora etapele incipiente ale dezvoltării descriptorilor de culoare și texturi, urmărind originile acestor tehnici. Ei discută despre metodele timpurii care se bazează pe histogramme simple de culoare și măsuri statistice pentru a reprezenta și compara culorile și texturile. Aceste abordări au pus bazele progreselor ulterioare în domeniu. Pe măsură ce domeniul a progresat, autorii evidențiază apariția unor descriptori de culoare și textură mai sofisticati, cum ar fi modelele binare locale (LBP). Cu toate acestea, odată cu apariția tehnicilor de învățare profundă, peisajul recunoașterii vizuale a suferit o transformare semnificativă. Autorii subliniază rolul esențial jucat de rețelele neuronale convoluționale (CNN) în revoluționarea domeniului. CNN-urile au demonstrat capacități excepționale în învățarea automată a caracteristicilor discriminatorii din seturi de date de imagini la scară largă, depășind performanța descriptorilor tradiționali manuali. Această trecere către abordări bazate pe învățarea profundă nu numai că a îmbunătățit acuratețea recunoașterii culorilor și texturii, dar a permis și dezvoltarea unor sisteme de recunoaștere vizuală mai complexe, capabile să gestioneze scenarii diverse și dinamice din lumea reală.

O lucrare diferită însă cu o relevanță mare în domeniu ar fi "Automatic Classification of Colour Fundus Images for Prediction Eye Disease Types Based on Hybrid Features" [6] publicată de Ahlam Shamsan, Ebrahim Mohammed Senan și Hamzeh Salameh Ahmad Shatnawi, apărută în varianta sa finală în data de 11 mai 2023. Acest articol se concentrează în principal pe clasificarea seturilor de date privind bolile oculare folosind tehnici hibride bazate pe extracția caracteristicilor prin metode de fuziune. "Colour fundus photography" (CFP) este o tehnică eficientă de examinare a ochiului și există o nevoie recunoscută pentru o tehnică automată de diagnosticare asistată de computer pentru a ajuta la diagnosticarea bolilor oculare. De aici putem deduce importanța detectării corecte a culorii. Autorii încep prin a evidenția impactul bolilor oculare la nivel mondial, subliniind necesitatea detectării unui diagnostic precis pentru a asigura un tratament în timp util și eficient. Ei recunosc provocările cu care se confruntă profesioniștii din domeniul sănătății în analiza manuală a unor volume mari de imagini, care pot consuma mult timp și pot fi supuse erorilor umane. Autorii propun o abordare nouă care combină diferite tehnici de extracție a caracteristicilor pentru a clasifica automat diferite tipuri de boli oculare pe baza caracteristicilor de culoare. Această cercetare stabilește scena pentru progrese ulterioare în tehnicile de diagnosticare asistate de computer, oferind perspective promițătoare pentru îmbunătățirea asistenței medicale oculare la nivel mondial.

"Fuzzy Logic-Controlled 6-DOF Robotic Arm Color-based Sorter with Machine Vision Feedback" [7] demonstrează aplicarea unui controler comun bazat pe logica fuzzy (FLJC - Fuzzy Logic-Based Joint Controller) ca sistem de sortare pe bază de culoare la un braț robotic. Este o lucrare publicată în (*IJACSA*) *International Journal of Advanced Computer Science and Applications*, Vol. 9, No. 5, 2018 și concepută de inginerii și electroniștii Alexander C. Abad, Dino Dominic Ligutan, Elmer P. Dadios, Levin Jaeron S. Cruz, Michael Carlo D.P. Del Rosario și Jho Nathan Singh Kudhal. Logica fuzzy este un domeniu matematic care se ocupă de impreciziile și incertitudinile care apar atunci când luăm decizii. Cercetarea prezintă un sistem de viziune artificială care poate distinge diferite

culori ce este integrat în brațul robotizat echipat cu FLJC. Sistemul de viziune artificială folosește un computer și o cameră Kinect pentru a extrage obiecte și coordonatele acestora pentru a ajuta dispozitivul de prindere. Utilizatorul poate controla procesul de sortare folosindu-se de interfața grafică. Rezultatele experimentale prezentate în lucrare prezintă eficacitatea și eficiența sistemului propus. Sistemul de sortare bazat pe culori a demonstrat o mare acuratețe în identificarea și sortarea obiectelor de diferite culori, prezentând potențialul aplicării sale practice în diverse industrii, cum ar fi producția, logistica și reciclarea. Cercetarea contribuie la avansarea sistemelor robotizate și a tehnologiilor de automatizare, permițând procese de sortare mai eficiente și mai fiabile. Cercetarea subliniază importanța luării în considerare atât a componentelor hardware, cât și a componentelor software în dezvoltarea sistemelor de recunoaștere a culorilor. Acest studiu deschide calea pentru noi progrese în automatizarea robotică și contribuie la progresul continuu al tehnologiilor de sortare în diverse industrii. Cea mai relevantă diferență dintre această lucrare și CSort, proiectul realizat de mine, este că aceasta lucrare recunoaște culoarea folosind software ca și metodă în timp ce CSort este bazat pe componente hardware reușind distingerea culorii prin modulul camera de la Raspberry Pi.

Lucrarea "Intelligent Color Based Object Sorting Using Robotics" [8] face parte dintr-o serie de publicații *International Journal of Robotics Research and Development (IJRRD)* și a fost publicată în iunie 2019. Lucrarea concepută de Rani A. Magadum și Uttam L. Bombale este o cercetare privind utilizarea roboticii pentru sortarea inteligentă a obiectelor bazate pe culori. Scopul acestui studiu este de a automatiza procesele de sortare în diferite industrii, cum ar fi sortarea legumelor, fructelor și ambalajelor. Se ridică problema sistemelor existente ce la momentul respectiv nu erau precise sau eficiente în timp. Studiul propune o soluție care folosește inteligența bazată pe senzori pentru recunoașterea culorilor. Sistemul identifică culorile obiectelor folosind procesarea imaginilor și senzorii de recunoaștere a culorilor pentru a identifica culorile găleților de colectare. Unul dintre avantajele cheie ale sistemului inteligent de sortare bazat pe culoare este adaptabilitatea acestuia la diferite obiecte și variații de culoare. Sistemul este capabil să recunoască o gamă largă de culori și poate fi programat pentru a gestiona modele sau combinații de culori specifice. Această flexibilitate permite aplicarea sa în diverse industrii care necesită o sortare precisă și eficientă, cum ar fi instalațiile de reciclare, fabricile de procesare a alimentelor și unitățile de producție. Descoperirile acestui studiu au implicații pentru diverse industrii, oferind o productivitate îmbunătățită, precizie și eficiență a costurilor. Lucrarea îmbunătățește acuratețea și eficiența procesului de sortare.

Luke Fina, Tabatha Mascarenhas, Cody Smith și Hakki Erhan Sevil de la "University of West Florida" concep în mai 2021 în cadrul *34th Florida Conference on Recent Advances in Robotics* lucrarea "Object Detection Accuracy Enhancement in Color based Dynamic Sorting using Robotic Arm" [9] proiectul ce utilizează brațul unui robot pentru a îmbunătăți acuratețea de detectare a obiectelor într-un sistem de sortare dinamică bazat pe culoare. Brațul robotizat poate recupera obiectele de pe banda transportoare în mișcare și le sortează în coșuri în funcție de culorile lor folosind algoritmul pe care l-a creat. Studiul îmbunătățește acuratețea sortării folosind tehnici de viziune computerizată sofisticate, cum

ar fi detectarea obiectelor folosind TensorFlow. Unul dintre aspectele notabile ale acestui studiu este accentul pus pe algoritmi de sortare bazați pe culoare. În timp ce multe proiecte existente utilizează algoritmi generici, cercetătorii și-au dedicat eforturile pentru a crea un algoritm personalizat care profită de caracteristicile și provocările unice asociate sortării pe culori. Acest algoritm personalizat asigură un grad mai mare de acuratețe și eficiență în distingerea și sortarea obiectelor în funcție de culorile lor. Prin teste riguroase, aceștia au demonstrat eficacitatea abordării lor în detectarea și sortarea cu precizie a obiectelor în timp real. Rezultatele au evidențiat îmbunătățiri semnificative ale preciziei de sortare în comparație cu metodele convenționale, evidențiind potențialul sistemului propus pentru diverse aplicații industriale. O asemănare între această cercetare și CSort ar fi folosirea camerei ca și componentă hardware pentru detectarea culorii, iar constatările acestui studiu deschid posibilități pentru procese de sortare industrială mai eficiente și mai fiabile, contribuind în cele din urmă la creșterea productivității și la reducerea cerințelor de muncă manuală.

3. TEHNOLOGII FOLOSITE

3.1. Raspberry Pi

Raspberry Pi este un microcomputer interoperabil cu orice dispozitiv hardware de intrare și ieșire transformând eficient configurația într-un PC complet la un cost redus [10]. Acesta permite persoanelor din diferite medii și niveluri de expertiză să experimenteze și să învețe despre calculatoare. Pentru a configura placa de dezvoltare trebuie introdus un card SD în spațiul prevăzut deoarece Raspberry Pi nu are memorie internă ceea ce este esențial pentru a rula un sistem de operare și pentru a putea utiliza și programa microcomputerul, este de asemenea esențial să avem un sistem de operare. Cardul SD servește ca opțiune de stocare principală pentru rularea și stocarea unui sistem de operare. Cu slotul dedicat pentru card SD de pe microcomputer, utilizarea unui card SD rămâne cea mai simplă și mai accesibilă opțiune. Permite utilizatorilor să instaleze și să pornească cu ușurință un sistem de operare pe Raspberry Pi, oferind un mediu de calcul stabil și personalizabil. Utilizarea unui card SD oferă, de asemenea, flexibilitate, deoarece pot fi instalate și comutate diferite sisteme de operare în funcție de nevoile și preferințele utilizatorului.

Odată ce cardul SD este introdus și sistemul de operare este instalat, microcalculatorul Raspberry Pi poate fi conectat la diferite periferice, cum ar fi un monitor, tastatură și mouse. Aceste periferice permit utilizatorilor să interacționeze cu microcalculatorul, să efectueze sarcini și să ruleze diferite aplicații software. Versatilitatea și accesibilitatea Raspberry Pi îl fac o alegere populară pentru diverse proiecte, de la automatizarea casei și centre media până la robotică și inițiative educaționale.

Dezvoltarea folosind Raspberry Pi este sprijinită în mod activ de o comunitate vastă de utilizatori și oferă documentație extinsă, tutoriale și forumuri în care persoanele își pot împărtăși proiectele, pot căuta asistență și pot face schimb de idei. Acest mediu de colaborare încurajează inovația și creativitatea, permițând utilizatorilor să exploreze întregul potențial al microcalculatorului și să își dezvolte abilitățile în programarea computerelor, electronică și rezolvarea problemelor.

Sistemul de operare folosit pentru Raspberry Pi se numește Raspbian [11]. Acesta este un sistem de operare gratuit, bazat pe Debian, acesta din urmă este bazat pe Linux. Raspbian oferă o interfață ușor de utilizat și o gamă largă de software preinstalat, ceea ce îl face o alegere populară pentru utilizatorii Raspberry Pi. Oferă un mediu familial pentru cei familiarizați cu ecosistemul Linux și oferă o bază solidă pentru rularea diferitelor aplicații și proiecte. Raspbian oferă documentație și resurse extinse, inclusiv ghiduri oficiale, forumuri și asistență din comunitate. Această mulțime de informații le permite utilizatorilor să rezolve problemele, să învețe noi abilități și să exploreze întregul potențial al dispozitivelor lor Raspberry Pi. Acesta suportă o gamă largă de limbaje de programare

și instrumente de dezvoltare, permițând utilizatorilor să-și creeze propriile aplicații și proiecte

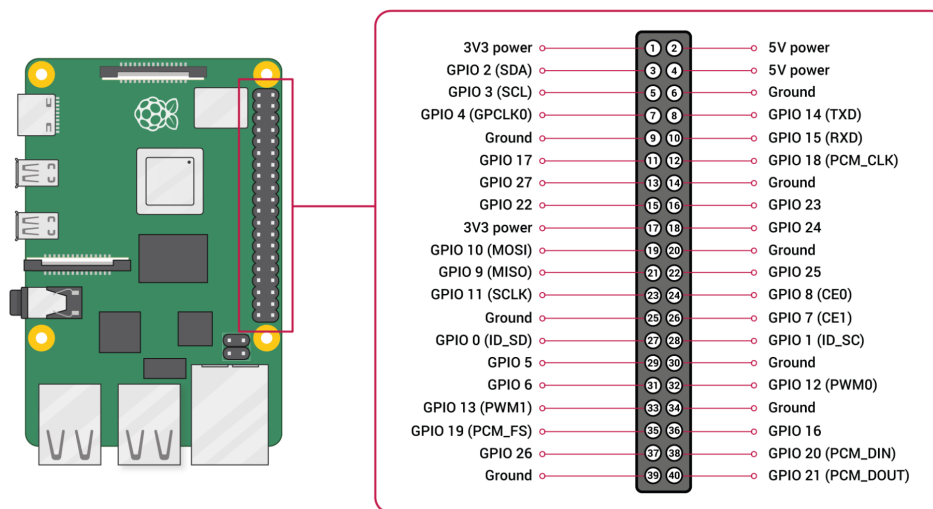
Linux este un sistem de operare care gestionează comunicarea între aplicațiile și hardware-ul computerului, a fost introdus în mijlocul anilor 1990 [12]. Se găsește în diverse dispozitive, de la smartphone-uri și electrocasnice până la supercomputere și servere de întreprindere. Linux este open-source, ceea ce înseamnă că resursele sau codul este disponibil gratuit pentru vizualizarea, modificare și distribuire de către oricine. Linux este format din mai multe componente care lucrează împreună pentru a crea un sistem de operare funcțional. Bootloader-ul este responsabil pentru gestionarea procesului de boot, asigurându-se că sistemul de operare se încarcă corect. La baza Linux se află nucleul, care acționează ca mijlocul sistemului prin gestionarea procesorului, memoriei și dispozitivelor periferice. Sistemul init, cum ar fi systemd, preia controlul după bootloader și inițializează spațiul utilizatorului în timp ce gestionează serviciile de fundal. Daemon-urile sunt servicii de fundal care se ocupă de sarcini precum imprimarea, gestionarea sunetului și programarea, începând fie în timpul procesului de pornire, fie la conectarea la desktop. Serverul grafic, cunoscut și sub numele de server X sau X, este responsabil pentru afișarea graficelor pe monitor. Mediul desktop oferă interfața utilizatorului pentru interacțiunea cu sistemul de operare, oferind diverse stiluri și caracteristici. Linux oferă o gamă diversă de medii desktop, inclusiv GNOME, KDE, Xfce și multe altele. În plus, Linux oferă o selecție vastă de aplicații software de înaltă calitate, care pot fi instalate și gestionate cu ușurință prin instrumente centralizate, cum ar fi magazinele de aplicații.

Un sistem de operare este un set de programe și utilități de bază care permit funcționarea și gestionarea eficientă a unui dispozitiv Raspberry Pi.

Printre caracteristicile unui Raspberry Pi se întâlnesc CPU (Central Process Unit), porturi HDMI, GPU (Graphic Processing Unit) ce îmbunătățesc capacitățile de procesare grafică ale dispozitivului, memorie RAM care oferă stocare temporară pentru o procesare rapidă și eficientă a datelor. Memorie RAM suficientă permite performanțe optime, asigurând aplicațiile și sistemul de operare să funcționeze fără probleme. Putem menționa și portul de ethernet cu ajutorul căruia se facilitează conectarea la rețelele locale sau la internet. Acest lucru deschide posibilități de accesare al resurselor externe cum ar fi servere sau alte dispozitive conectate la rețea, astfel Raspberry Pi devine o platformă versatilă pentru proiecte care necesită comunicare și conectivitate. În continuare slotul pentru card SD extinde capacitățile de stocare, iar GPIO sunt componente esențiale ale dispozitivului Raspberry Pi, deoarece contribuie la diversitatea aplicațiilor sale. Pinii GPIO sunt utilizați pentru a interacționa cu alte circuite electronice. Aceștia pot citi și controla semnalele electrice de la alte plăci sau dispozitive, în funcție de modul în care utilizatorul îi programează.

În Figura 2 se observa pinout-ul plăcuței Raspberry Pi. Pinii plăcuței sunt folosiți pentru conectarea servomotorului. Se folosesc pinii 4, 6 și 11 și restul sunt nefolosiți.

Figura 2 - Pinout Raspberry Pi



Diferitele tipuri de pini, numerotarea și caracteristicile lor se pot vedea în Tabelul 2.

Tabelul 2 - Aranjarea pinilor pe Raspberry Pi

Tip pin	Numerotare pini	Descriere
3.3V (Volți) alimentare	1, 17	Asigură o sursă de alimentare de 3,3 volți, care este utilizată pentru alimentarea perifericelor și a senzorilor de nivel scăzut
5V (Volți) alimentare	2, 4	Asigură o sursă de alimentare de 5 volți, care este utilizată pentru alimentarea componentelor externe care necesită o tensiune mai mare
Pini de împământare	6, 9, 14, 20, 25, 30, 34, 39	Acționează ca masă de referință și cu scopul de a conduce curentul electric să curgă înapoi la sursa de alimentare
GPIO - Intrare/ieșire de uz general	3, 5, 7, 8, 10, 11, 12, 13, 15, 16, 18, 19, 21, 22, 23, 24, 26, 27, 28, 29, 31, 32, 33, 35, 36, 37, 38, 40	Pot fi programați să funcționeze ca intrări sau ieșiri pentru a interfața cu diverse dispozitive externe
SPI (Serial Peripheral Interface) sau Interfață periferică serială	11, 12, 19, 21, 23, 24, 26, 35, 36, 38, 40	Permit comunicarea senzori, afișaje și cipuri de memorie

I2C (pinii circuitului inter-integrat)	3, 5, 27, 28	Permit comunicarea cu senzori, ADC-uri (Convertoare analog-digitale) și EEPROM-uri (Memorie de citire, programabilă)
UART (Universal Asynchronous Receiver-Transmitter) sau receptor-transmițător asincron universal	3, 5, 7, 8, 10, 11, 19, 21, 23, 24, 26, 27, 28, 29, 31, 32, 33, 36	Facilitează comunicarea în serie cu dispozitive precum modulele GPS, modulele Bluetooth și consolele seriale
PCM (Pulse-Code Modulation) sau modularea codului de impulsuri	12, 35, 38, 40	Permit Raspberry Pi să interfațeze cu dispozitive audio, cum ar fi convertoare digital-analog

Datorită acestor caracteristici, placa de dezvoltare ne permite integrarea cu ușurință a modului de cameră cât și a servomotorului. Pinii GPIO permit conectarea servomotorului la unitatea centrală a proiectului, adică Raspberry Pi, acesta din urmă fiind prevăzut cu spațiu special pentru conectarea modului de cameră, astfel fiind perfect pentru dezvoltarea proiectului realizat pentru recunoașterea culorii obiectelor și sortarea lor bazat pe culoare.

3.2. Python - OpenCV

Python este unul dintre cele mai populare limbaje de programare din lume și este folosit pentru diferite scopuri cum ar fi pentru a construi site-uri web și software, pentru a automatiza sarcini și pentru a efectua analize de date [13]. Python este un limbaj de uz general, ceea ce înseamnă că poate fi folosit pentru a crea o varietate de programe diferite și nu este specializat pentru probleme specifice. Este adesea folosit pentru dezvoltarea site-urilor web și software. Simplitatea asemănătoare cu sintaxa limbajului natural, îl face accesibil atât dezvoltatorilor, cât și non-dezvoltatorilor. Acesta este utilizat în numeroase domenii, inclusiv analiza datelor și deși este potrivit și începătorilor, acesta este o alegere și pentru persoane avansate, permițându-le acestora să efectueze calcule sau statistici complexe, ajută la manipularea datelor și chiar la construcția algoritmilor de învățare automată. Bibliotecile TensorFlow și Keras ajută și simplifică procesele ce țin de date. Cadrele Python precum Django și Flask facilitează crearea de aplicații web.

Redenumirea fișierelor, calcule și operații complexe sau lucrarea simultană cu mai multe fișiere face particularitatea de automatizare un adevărat ajutor. De asemenea, se poate folosi cu ușurință pentru testare sau creare de prototipuri permițând urmărirea erorilor. Python este un limbaj open-source, disponibil gratuit pentru utilizare și distribuție,

chiar și în scopuri comerciale. Folosind această combinație, pot ajuta proiectului meu să identifice și să diferențieze diferite culori în timp real.

Python vine preinstalat pe Raspbian și versiunea folosită în momentul actual este Python 3. Utilizând bibliotecile Python împreună cu OpenCV pot dezvolta cu ușurință un sistem de recunoaștere a culorilor. Python oferă instrumentele necesare pentru a procesa și analiza eficient datele vizuale. Am ales să folosesc acest limbaj de programare deoarece oferă biblioteci extinse, este ușor de folosit și oferă un suport bun pentru informațiile necesare. Se pot implementa cu ușurință funcționalități specifice de care eu am nevoie cum ar fi recunoașterea culorilor.

OpenCV este o librărie open-source compatibilă cu limbaje de programare cum ar fi Python sau C++ [14]. OpenCV permite construirea proiectelor de viziune computerizată și învățare automată. Oferă o gamă largă de funcții și algoritmi pentru manipularea imaginilor și video, detectarea obiectelor și urmărirea obiectelor, ceea ce face această librărie perfectă pentru detectarea culorii obiectelor, oferind această funcționalitate în mod simplificat. O altă caracteristică notabilă ar fi aceste analize realizate în timp real.

Cum am menționat anterior, sistemul de operare pentru Raspberry Pi (Raspbian) vine preinstalat cu Python, astfel adăugarea bibliotecii OpenCv este convenabilă, făcând mediul de lucru unul cât mai simplu. Urmând procesul de instalare, dezvoltatorii pot configura rapid mediul software necesar pentru a începe folosirea OpenCV în cadrul proiectelor Python.

În cadrul acestei lucrări este folosit OpenCv pentru avantajele enumerate, cum ar fi manipularea imaginilor și în final pentru a face posibilă funcționalitatea de recunoaștere a culorii folosind camera.

Combinarea Python și OpenCV oferă numeroase posibilități pentru analiza vizuală. Se poate implementa eficient algoritmi de procesare a imaginii și aplicat proiectului meu se pot recunoaște în mod cât mai simplu culorile obiectelor.

3.3. Android Studio

Android Studio este platforma oficială de dezvoltare integrată (IDE - Integrated Development Environment) pentru Android pentru crearea de aplicații [15]. Acesta oferă o gamă largă de funcții furnizate de IntelliJ IDEA cu scopul de a îmbunătăți productivitatea în crearea de aplicații Android. Ca și caracteristici specifice putem menționa că este un sistem bazat pe Gradle, care este flexibil pentru personalizarea aplicațiilor Android permite configurarea și extinderea ușoară a procesului de construire, permițând dezvoltatorilor să personalizeze și să extindă procesul de construire, să creeze mai multe APK-uri cu diferite caracteristici și să folosească codul și resursele în seturi de surse. Emulatorul este rapid și dispune de numeroase funcții de testare și rularea aplicațiilor, de asemenea oferă o platformă unificată pentru dezvoltarea de aplicații care sunt compatibile cu diferite dispozitive Android. GitHub oferă șabloane de cod pre-construite și posibilitatea de a importa cod eșantion, facilitând dezvoltarea caracteristicilor comune ale aplicației, iar instrumentele de testare asigură fiabilitatea și calitatea aplicațiilor Android dezvoltate.

Android Studio este prevăzut cu suport C++ și NDK și are încorporat suport pentru platforma Google Cloud astfel se poate realiza cu ușurință integrarea serviciilor Google Cloud Messaging și App Engine în aplicațiile Android. Oferă o reprezentare vizuală a fișierelor de proiect și facilitează navigarea și accesul ușor la fișierele sursă cheie.

Android Studio include inspecții și validare a codului pentru a asigura calitatea și eficiența codului.

Datorită simplității pe care aplicația mobilă dezvoltată de mine o are, am ales Android Studio pentru a beneficia de mediul și instrumentele ușor de folosit care au ajutat la managerierea într-un mod cat mai bun al timpului de dezvoltare, totodată fiind ușor de conectat cu placa Raspberry Pi.

Ca și limbaj de programare folosit pentru dezvoltarea aplicației acestui proiect am folosit Java. Acesta este un limbaj de programare utilizat pe scară largă și o platformă de calcul devenind o componentă esențială industriei digitale [16]. A fost lansat în 1995 și este fundația multor servicii sau aplicații. Java este disponibil gratuit și se poate folosi de către dezvoltatori experimentați sau de către începători, putându-se accesa instrumente de dezvoltare. Regulat se recomandă actualizarea versiunii, pentru a beneficia de performanță.

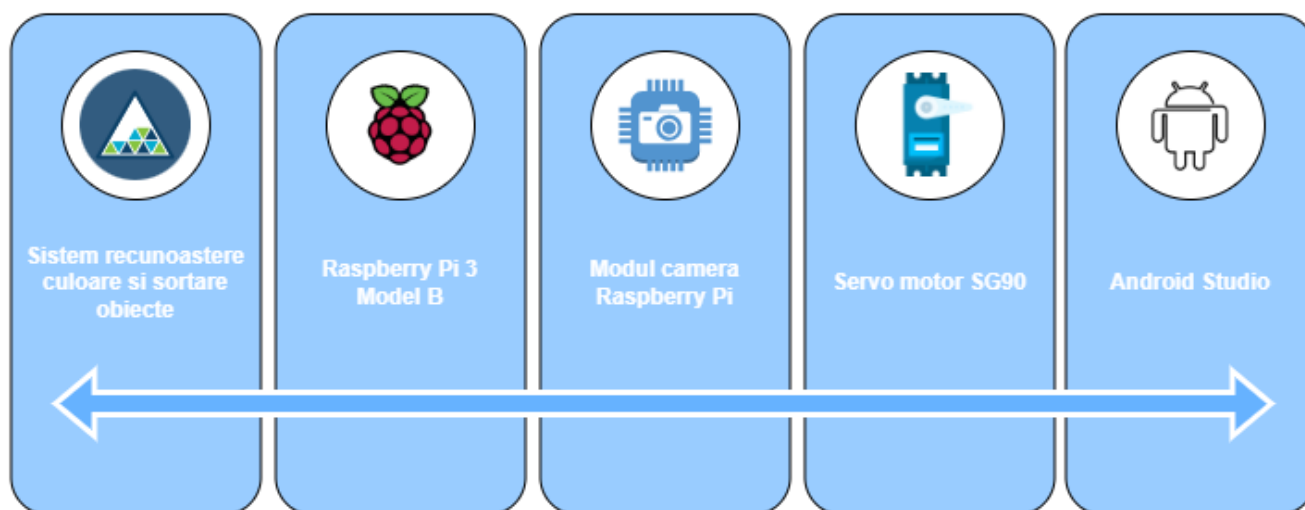
Motivul alegerii Java este asemenator cu cel anterior precizat, anume faptul că oferă o gamă largă de servicii. Java poate rula pe diverse sisteme de operare și dispozitive și dispune și de librării vaste, este compatibil cu Android Studio, are o performanță și eficiență ridicată și dispune de biblioteci variate, motiv pentru care consider o alegere potrivită pentru aplicația Android pe care eu o dezvolt.

4. ARHITECTURA PROIECTULUI

4.1. Arhitectura sistemului

Sistemul creat este unul ce va recunoaște culoarea obiectelor ce trec prin fata camerei, și pe baza informațiilor primite, adică diferite culori, va sorta obiectele. Sortarea se face prin rotirea paletelor unui servomotor la stânga sau la dreapta, având ca scop blocarea și deblocarea canalelor create în sistemul fizic. Toate aceste functionalitati vor fi pornite și oprite de către utilizator cu ajutorul unei aplicații mobile Android. Arhitectura generală a sistemului se poate vedea în Figura 3 și reprezintă o privire de ansamblu a proiectului.

Figura 3 - Arhitectura generală a sistemului



Componenta principală a sistemului este placa de dezvoltare Raspberry Pi. Modulul camerei și servomotorul sunt conectate la placă și îndeplinesc principalele roluri în proiect. Camera ajută la detectarea culorilor iar servomotorul ajută la rotirea unei palete ce este atașată sistemului pentru a servi la sortare. Aplicația realizată în Android Studio ușurează interacțiunea utilizatorului cu proiectul, oferă suport vizual și permite utilizatorilor să interacționeze cu procesul de sortare. Informațiile de culoare capturate sunt procesate și analizate în timp real, permițând sistemului să ia decizii instantanee cu privire la mecanismul de sortare adecvat. Servomotorul, controlat de Raspberry Pi, rotește cu precizie paleta de sortare, asigurându-se că fiecare obiect este îndreptat către zona corectă desemnată în funcție de culoare.

Mișcarea paletei atașate sistemului va bloca și debloca accesul obiectelor în stânga sau dreapta pentru a se realiza sortarea. După ce camera va recepta o culoare, o va transmite la microcalculator, acesta din urmă va trimite mai departe acest semnal

servomotorului. Raspberry Pi efectuează toată logica și ajută la sincronizarea componentelor de bază. Camera se comportă ca o sursă de intrare prin receptarea culorii și trimite în timp real culorile. Servomotorul este componenta de ieșire ce primește de la plăcuță semnale în funcție de care realizează rotație la dreapta sau la stânga. La fel ca și camera, servomotorul reacționează în timp real la informația primită. Acest sistem se completează perfect și lipsa oricăreia dintre componente ar duce la eșuarea scopului propus.

Toate aceste funcționalități pot fi declanșate de către utilizator folosind butoanele din aplicația Android. Fără această aplicație rularea sistemului ar fi posibilă doar prin conectarea plăcuței Raspberry Pi la periferice, cum ar fi un monitor. Astfel pentru a utiliza cu ușurință sistemul, aplicația dezvoltată în Android Studio este esențială.

4.2. Arhitectura hardware

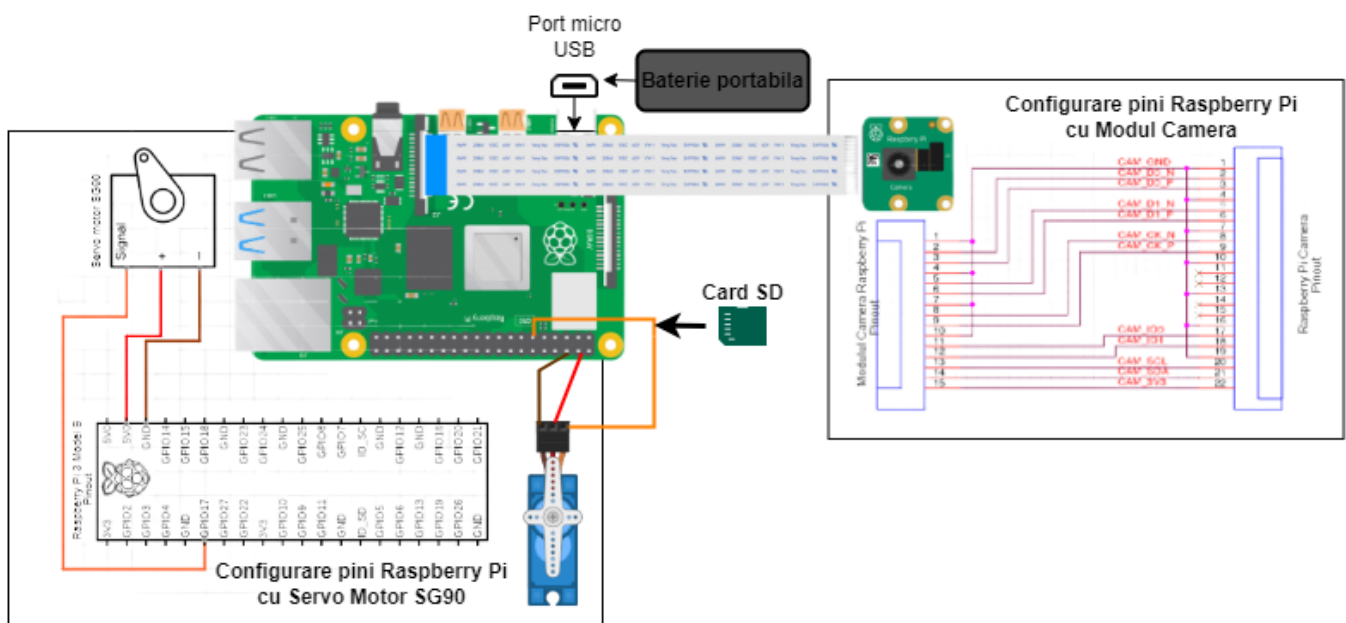
Placa de dezvoltare Raspberry Pi 3 Model B este principala componentă a sistemului. Pentru a programa pe această plăcuță am folosit un card SD pe care am descărcat sistemul de operare Raspbian. Folosind terminalele USB am conectat tastatura și mouse cât și HDMI, pentru conectarea monitorului, am scris codul și am programat plăcuța, am ales această variantă pentru a avea control deplin și a dezvolta fără întreruperi. Această configurare a facilitat un mediu de programare convenabil și ușor de utilizat. Fiind versatilă, placa de dezvoltare este programabilă și se poate conecta cu ușurință la celelalte componente.

Alimentarea pe parcursul dezvoltării aplicației a fost făcută utilizând portul micro USB în care s-a introdus alimentatorul care a primit energie de la priză. Am ales această opțiune pentru a asigura o sursă sigură de producere a energiei. Acest lucru a permis aplicației să ruleze fără întreruperi și să beneficieze de o sursă de alimentare stabilă. Pentru utilizarea cât mai ușoară a proiectului se folosește o baterie externă ce este de asemenea conectată la plăcuță prin portul micro USB, aceasta permite transportarea și utilizarea independent de o sursă fixă de alimentare. Utilizatorii se pot bucura de mobilitate și pot folosi aplicația oriunde doresc, fără restricții legate de disponibilitatea unei prize. Când conectăm bateria la placa Raspberry Pi, aceasta reglează curentul care ajunge la microcalculator, asigurând o cantitate stabilă de curent. Această caracteristică este deosebit de utilă pentru a evita fluctuațiile curentului și pentru a proteja placa de supratensiune potențială sau căderi bruște de tensiune. Bateria folosită are o ieșire de 5V (volți), în momentul conectării celor două, bateria reglează, așa cum am menționat, curentul ce ajunge la Raspberry Pi, astfel placa va primi o cantitate stabilă de curent. Acest nivel de tensiune este compatibil cu cerințele de putere ale plăcii, asigurând o funcționare optimă și stabilă. Bateria ajută sistemul să funcționeze independent pentru o anumită perioadă de timp, când bateria se descarcă, adică va rămâne fără energie. Procesul de încărcare se poate face folosind un încărcător sau prin conectarea bateriei la o sursă de alimentare. Este important de reținut că procesul de încărcare al bateriei este

independent de funcționarea aplicației, permițând utilizatorilor să continue să utilizeze proiectul chiar și în timpul procesului de încărcare.

Pentru exemplificarea conexiunilor realizate în acest proiect și pentru a avea o reprezentare vizuală ne vom folosi de Figura 4. Aici se pot observa cele două componente principale ce sunt atașate plăcuței de dezvoltare Raspberry Pi cât și legăturile dintre ele.

Figura 4 - Arhitectura hardware a sistemului



Se poate observa că pe lângă servomotorul SG90, care este conectat la pinii 4, 6 și 11, camera se conectează la portul încorporat și dedicat folosirii modului de cameră. Pinii menționați sunt desemnați pentru controlul servomotorului și permit o poziționare precisă pe baza semnalelor primite. Prin servomotor aplicația poate efectua diverse acțiuni sau mișcări mecanice conform instrucțiunilor programate. Bateria portabilă se conectează prin portul microUSB și are ca scop alimentarea cu energie a plăcuței. Cardul SD îndeplinește scopul său principal și anume acela de a stoca sistemul de operare, acesta permite funcționarea și controlul tuturor componentelor și caracteristicilor aplicației.

Pentru a asigura conectivitatea și transferul de date între placa Raspberry Pi și alte componente, cum ar fi servomotorul și camera, sunt utilizate diferite porturi și interfețe. Astfel această arhitectură asigură că funcționalitățile necesare să fie prezente, sunt o soluție potrivită pentru a atinge obiectivul setat.

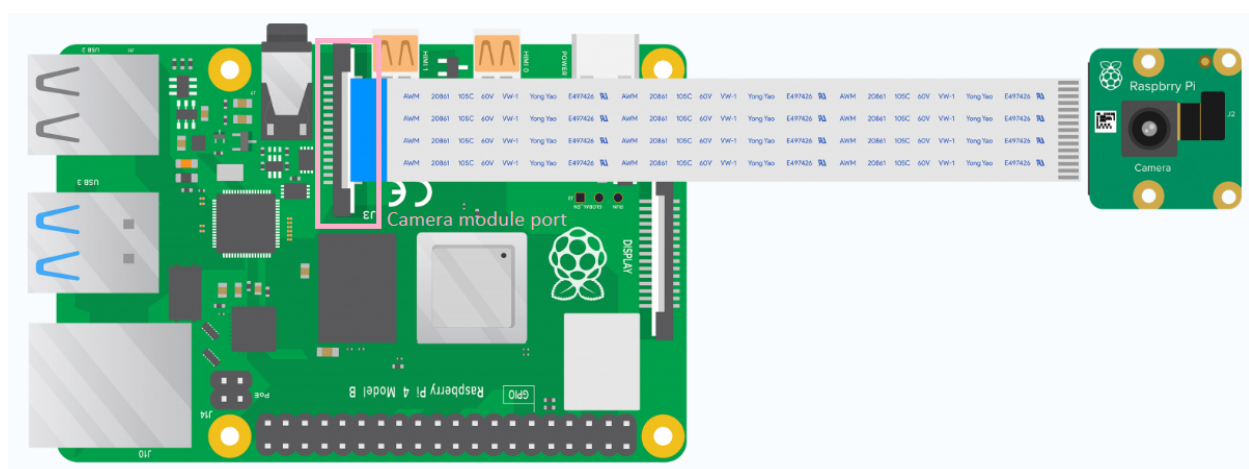
4.2.1. Modul Cameră Raspberry Pi

Printre accesoriile compatibile cu placa Raspberry Pi se găsește și modulul pentru cameră. Camera este o componentă de intrare ce începe să funcționeze în același timp cu

rularea programelor Python oprește odată cu acestea. Pornirea și oprirea se realizează de către utilizator folosind interfața grafică a aplicației Android. Camera funcționează în sincronizare cu sistemul.

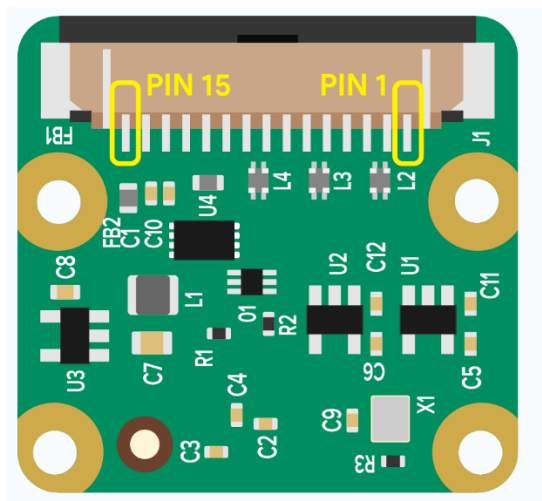
În acest proiect camera are ca scop urmărirea obiectelor ce sunt în fața acesteia și să aștepte să apară în vizor un obiect ce are una din culorile necesare pentru a trimite semnal mai departe și să pornească servomotorul. Placa are încorporat portul pentru modulul camerei [17] și este prezentă pe placuta ca ca în Figura 5, acest port dedicat facilitează conectarea și integrarea ușoară a modulului camerei în sistem.

Figura 5 - Portul pentru modulul camerei la Raspberry Pi



Există bineînțeles mai multe tipuri de camere, cele mai uzuale sunt cele cu 15 pini și cu 22 de pini. Pentru seriile A și B de Raspberry Pi se folosește conectorul cu 15 pini [11]. Utilizarea unui conector standardizat cu 15 pini asigură compatibilitatea și integrarea ușoară a modulului camerei cu placa Raspberry Pi. Acest conector urmează un standard industrial adoptat pe scară largă, permițând o conectivitate fără întreruperi și o dezvoltare simplificată a proiectelor bazate pe camere. În Figura 5 se poate observa cum camera este conectată la plăcuță folosind un cablu standard 15 pini - 15 pini FFC (flexible flat cables) [18]. Acest design de cablu plat și flexibil permite rutarea și instalarea ușoară, asigurând o conexiune fiabilă fără a compromite constrângerile de spațiu ale sistemului. Pinii au un rol esențial în stabilirea conexiunilor electrice corecte și a transferului de date între cameră și placă. Fiecare pin are o funcție specifică și este atribuit pentru a îndeplini o anumită sarcină, variind de la sursa de alimentare și conexiunile la pământ până la transmisia de date și semnalele de control. Cei 15 pini de pe cameră au structura ca în Figura 6. Aranjarea pinilor pentru modulul camerei este prezentată în Tabelul 3.

Figura 6 - Aranjarea pinilor pe camera Raspberry Pi



Tabelul 3 oferă o imagine de ansamblu cuprinzătoare a aranjamentului pinilor pentru modulul camerei menționându-se numerele de pin, funcțiile corespunzătoare și descrierile acestora. Aceste informații sunt esențiale pentru dezvoltatorii și utilizatorii care trebuie să înțeleagă pinout-ul și să utilizeze eficient modulul camerei în cadrul proiectelor lor.

Tabelul 3 - Aranjarea pinilor pentru modulul camerei pe Raspberry Pi

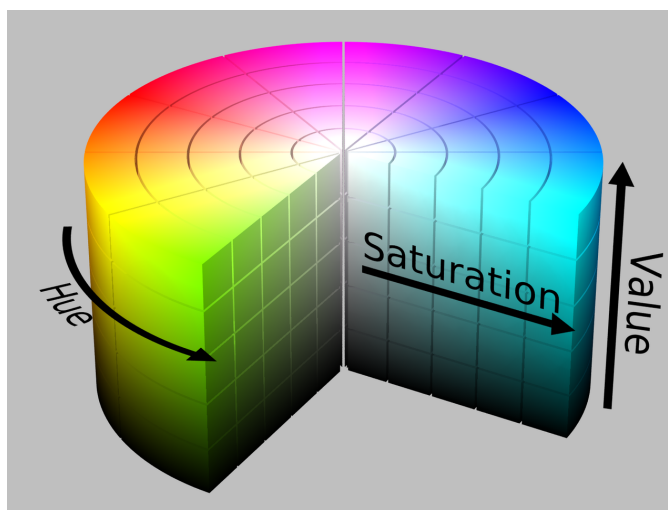
Pin #	Nume	Descriere
1	GND	Ground
2	CAM_D0_N	MIPI Data Lane 0 Negative
3	CAM_D0_P	MIPI Data Lane 0 Positive
4	GND	Ground
5	CAM_D1_N	MIPI Data Lane 1 Negative
6	CAM_D1_P	MIPI Data Lane 1 Positive
7	GND	Ground
8	CAM_CK_N	MIPI Clock Lane Negative
9	CAM_CK_P	MIPI Clock Lane Positive
10	GND	Ground
11	CAM_IO0	Power Enable
12	CAM_IO1	LED Indicator

13	CAM_SCL	I2C SCL
14	CAM_SDA	I2C SDA
15	CAM_3V3	3.3V Power Input

În cadrul acestei lucrări camera este folosită pentru detecția culorii.

Recunoașterea culorilor se face încadrând culorile obiectelor în limite de HSV [19]. Hue adică nuanța se bazează pe cele trei culori primare, roșu, galben și albastru și pe cele trei secundare, adică portocaliu, verde și violet. Cu cât saturația unei culori este mai mare, cu atât aceasta este mai vie și mai intensă. Cu cât saturația este mai mică, cu atât este mai aproape de gri pur pe scara de gri. Value sau valoarea unei culori se referă la nivelul de luminozitate al acesteia. Un avantaj al modelului HSV este că permite o modificare ușoară a proprietăților culorii. De exemplu, ajustarea nuanței permite tranziții ușoare între diferite culori, în timp ce modificarea saturației modifică intensitatea culorii. Schimbarea valorii oferă control asupra luminozității, și permite utilizatorilor să creeze efecte de umbră sau să simuleze variațiile de iluminare.

Figura 7 - Reprezentarea HSV



Acest model este util în simularea modului în care culorile apar în diferite condiții. Modelul se bazează pe principiul că schimbarea componentei valorice reprezintă efectul de adăugare sau eliminare de lumină.

În realizarea acestui proiect modelul se va folosi pentru a ajuta la detecția culorii prin încadrarea obiectelor percepute de cameră între anumite limite HSV. Acest lucru se va prezenta pe larg în capitolul Implementare.

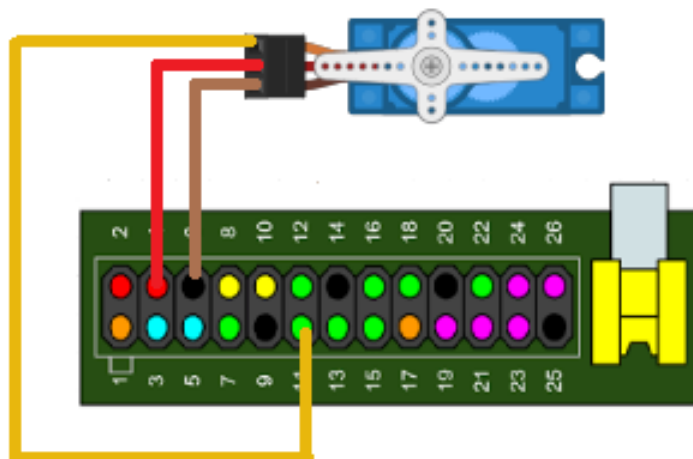
4.2.2. Servomotor SG90

Servomotorul SG90 cântărește 9 grame, are dimensiuni mici și este ușor, fiind recunoscut pentru aceste caracteristici care-l fac ușor de integrat în diferite proiecte. Deși dispune de această formă compactă, are putere mare de ieșire. Se poate roti aproximativ 180 de grade, 90 de grade în stânga și 90 de grade în dreapta, funcționează la fel ca cele standard. Pentru a-l utiliza și controla se poate utiliza orice cod servo, hardware sau bibliotecă. Astfel fiind flexibil, acesta poate realiza mișcări fără a fi necesară construirea unui controller de motor, este de asemenea o alegere buna pentru acest proiect deoarece este de dimensiuni mici. Acesta dispune de 3 brațe [20]. În proiectul realizat se folosește un singur braț, acesta având două palete, așezate opus și formând un unghi de 180 grade. Brațele au un rol esențial în această lucrare, ele miscand obiectul ce blochează și deblochează canalele pe care obiectele se sortează în funcție de culoare.

Viteza acestuia este de 0.1 s/60 grade, servomotorul oferă o reacție instantă la semnale, asigurând mișcări precise și prompte. Este prevăzut cu 3 fire pentru conectare, culorile acestora fiind specifice, având pentru 5V (rosu), împământare (maro) și semnal (galben). Conectarea la Raspberry Pi se face pentru acest proiect la pinii 4,6 respectiv 11. Ținând cont de pinout-ul placutei pinul 4 reprezintă 5V, pinul 6 este masa (GND) și pinul 11 este GPIO 17.

GPIO 17 funcționează conform schemei de numerotare WiringPi, care este o bibliotecă software ce simplifică interacțiunea cu Raspberry Pi. Folosește propria schemă de numerotare a pinilor și are ca scop oferirea unei interfețe constante printre mai multe limbaje de programare [21]. Practic numerotarea fizica a placutei, în cazul nostru pinul 11 corespunde numerotării WiringPi, adică GPIO 17. În Figura 8 este exemplificată conexiunea anterior menționată.

Figura 8 - Conexiune Raspberry Pi - servomotor



Poziția este controlată prin trimiterea de impulsuri de lățime variabilă (PWM) către servomotor. Lățimea impulsului de 1,5 milisecunde (ms) este echivalentă cu poziția „0”. Aceasta este poziția centrală a servomotorului. Poziția „90” indică o lățime de impuls de aproximativ 2 milisecunde (ms). Poziția „-90” indică o lățime de impuls de aproximativ 1 milisecundă (ms). Varierea lățimii impulsului permite controlarea poziției și mișcării servomotorului, astfel se poate modifica direcția de rotire dar și gradele cu care se rotește.

Datorită eficienței energetice cat si capacităților de poziționare precise servomotorul SG90 ajută proiectul meu fiind componenta principală a părții de sortare, fără de care acest proiect nu are o funcționalitate practică. Dimensiunea sa mică îl face ușor de integrat în proiect. În plus, consumul său mic de energie prelungește timpul de funcționare, mai ales în cazul actual în care se va folosi o baterie externă pentru a da energie plăcuței.

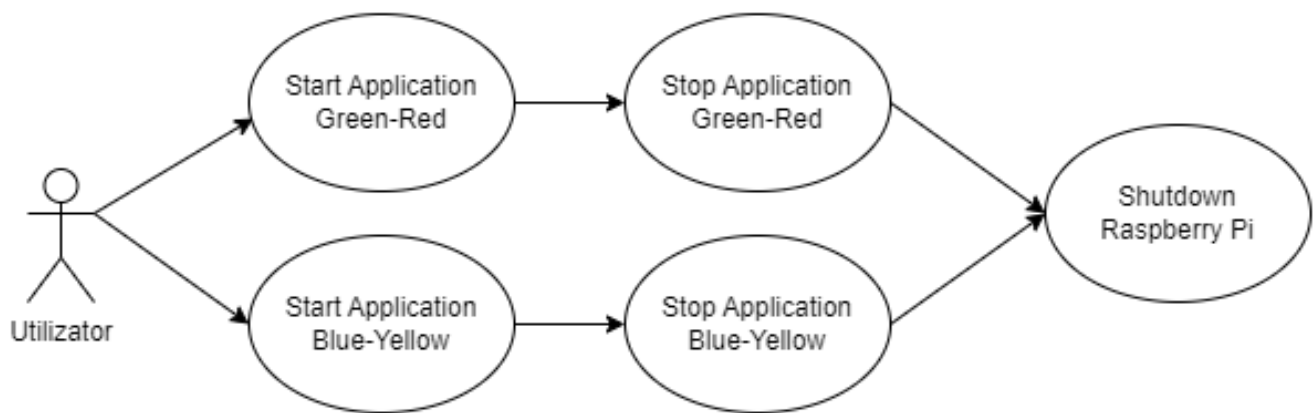
Mecanismul de sortare din proiectul meu va putea manipula obiectele în funcție de culoarea lor cu precizie, folosind eficiența și acuratețea servomotorului SG90. Capacitatea de a gestiona poziția și mișcarea servomotorului prin utilizarea semnalelor PWM garantează un proces de sortare precis și sigur.

4.3. Arhitectura software

În acest capitol voi reprezenta arhitectura software a acestui proiect, aceasta constă în aplicația Android ce are ca scop interacțiunea utilizatorului cu sistemul de recunoaștere a culorii și sortare în funcție de aceasta.

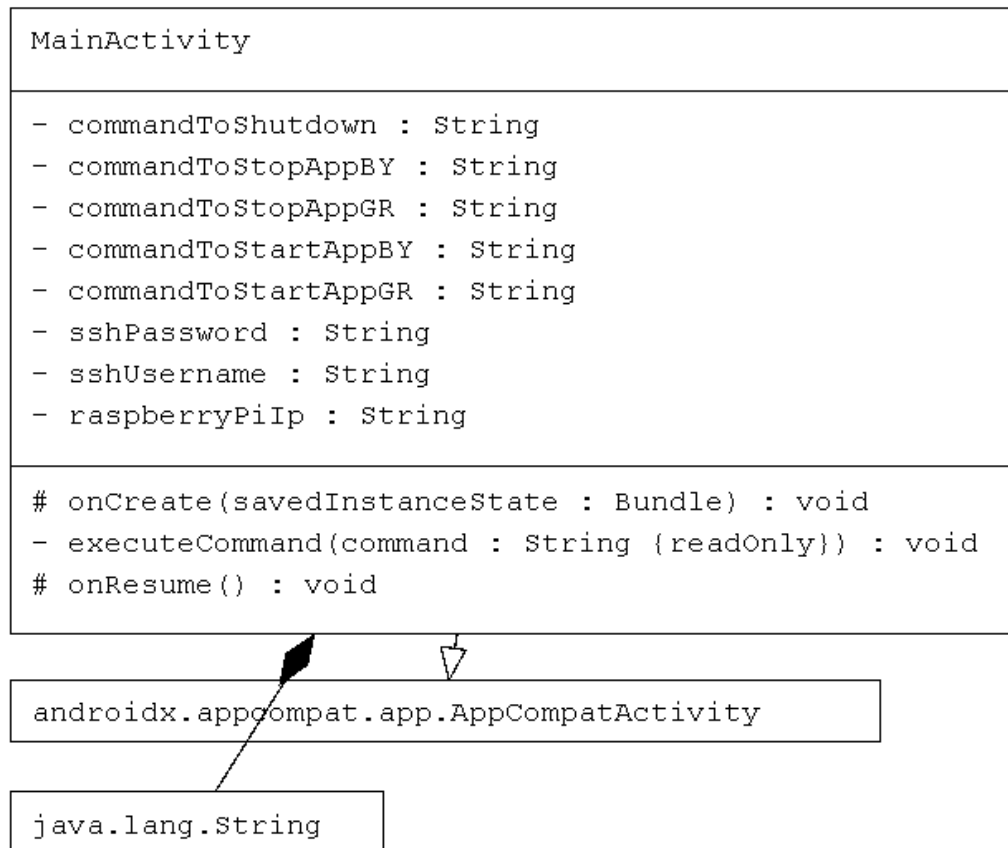
Pentru a reprezenta structura și comportamentul aplicației am realiza diagrama de UseCase. În Figura 9 sunt prezentate funcționalitățile aplicației. Un utilizator poate porni și opri două programe diferite: programul pentru recunoașterea și sortarea obiectelor de culoare roșie și verde și programul pentru recunoașterea și sortarea obiectelor de culoare galbenă și albastră. Acestea sunt cele patru funcționalități principale, cea de-a cincea având scopul de a opri alimentarea plăcuței Raspberry Pi prin butonul “Shutdown”, moment în care celelalte butoane nu vor mai funcționa până la alimentarea plăcuței cu energie. În Figura 9 funcționalitățile de pornire și oprire a celor două aplicații menționate anterior sunt reprezentate ca “Start Application Green-Red”, “StopApplication Green-Red”, “Start Application Blue-Yellow” și “Stop Application Blue-Yellow”.

Figura 9 - Diagrama UseCase



Folosindu-ne de diagrama UseCase am exemplificat reacția aplicației la stimulii externi, adică la utilizator. Urmează cu ajutorul diagramei UML sa discutăm despre structura internă a aplicației. În Figura 10 este ilustrată activitatea aplicației (MainActivity) unde sunt enumerate proprietățile clasei. Pe lângă cele ce ajung prin butoane la utilizator, adică cele 5 comenzi: oprire și pornire funcție de detectare și sortare obiecte roșii și verzi sau galbene și albastre, cat și “Shutdown” care va opri alimentarea cu energie a plăcuței, sunt prezentate și trei proprietăți esențiale și legate între ele. “sshPassword”, “sshUsername” și “raspberryPiIP” ajută la transmiterea de date între Raspberry Pi și aplicația Android. “sshPassword” și “sshUsername” sunt informații unice pe care plăcuța folosită de mine le are, sunt esențiale pentru a putea asigura legătura. În aceeași situație este și “raspberryPiIP” care se modifică în funcție de rețeaua la care este conectat, conexiunea este realizată doar în cazul în care ambele sunt conectate la aceeași rețea de internet.

Figura 10 - Diagrama UML



În Figura 10, pe lângă proprietățile descrise, se află și cele două metode aflate în aplicație: `onCreate()` și `onResume()`. În general o metodă este o parte de cod care realizează o acțiune specifică, ajută la o mai bună organizare și structurare a codului. În aceasta aplicație scopul metodelor este de a declanșa butoanele vizibile utilizatorilor.

Metoda `onCreate()` este apelată când o activitate este inițializată, aceasta apelează superclasa `onCreate()` pentru a se asigura că toate inițializările sunt făcute corespunzător. Referințele la butoane și numele lor sunt prezente în metodă. Pentru fiecare buton în parte se va executa o comandă specifică predefinită, acest lucru este posibil prin `executeCommand()`.

Metoda `onResume()` se comportă asemănător cu cea menționată anterior, apelează superclasa `onResume()` prin `“super.onResume()”` și se asigură de menținerea ciclului de viață al aplicației. Aceasta este apelată după start și înainte de pauză.

Metoda `onCreate()` apelează metoda `“executeCommand”`, cea din urmă asigură trimiterea semnalului către Raspberry Pi. Conexiunea se realizează prin SSH, acesta este un protocol de rețea care oferă acces securizat la computere prin rețele nesecurizate [22]. Permite autentificarea de la distanță, executarea comenzilor și transferul de fișiere. A fost

creat pentru a înlocui transferul de date nesigur, asigurand administrarea și conectarea la distanță între dispozitive.

Conexiunea propriu-zisă prin SSH a aplicației mobile și a plăcuței se realizează doar în cazul în care cele două sunt conectate la aceeași rețea. Adresa IP (protocolul de internet) este o marcă unică a fiecărui dispozitiv conectat la o rețea ce folosește protocolul de internet pentru a comunica și are scopul de a identifica în mod unic dispozitivul pentru a transmite sau primi date. A fost necesară găsirea IP-ului de conectare, apoi acesta a fost transmis aplicației mobile, și folosind portul 22 s-a realizat conexiunea între dispozitive. Pentru a asigura conexiunea dintre placă și aplicația de pe telefon, IP-ul trebuie modificat în funcție de rețeaua la care Raspberry Pi este conecta. De asemenea, portul SSH trebuie activat pe plăcuță pentru a fi posibilă conectarea.

Astfel pentru a asigura securitatea, transferul de informație, executarea comenzilor pentru Raspberry direct de pe telefon, având acces la distanță față de sistem, conectarea plăcuței prin SSH la aplicația mobilă este o alegere foarte bună pentru proiectul meu, îndeplinind nevoile de baza pentru sistem.

5. IMPLEMENTARE

În acest capitol voi prezenta codul cu ajutorul căruia am realizat întreg sistemul, bucăți de cod pentru părți diferite dar care comunică între ele și astfel se ajunge la un rezultat final. Voi discuta de asemenea despre cum se realizează comunicarea între componente și toate bibliotecile necesare și importate în proiect.

Fișierele prezente sunt două Python, create pentru recunoașterea culorilor roșu și verde respectiv galben și albastru. Avem de asemenea și fișierele pentru aplicația Android. Codul celor două fișiere Python pentru recunoașterea culorilor este identic în mare parte, având aceeași structură și realizând aceleași cerințe, având diferite specificații pentru culori, în funcție de acestea.

5.1. Algoritm recunoaștere culoare obiecte

Recunoașterea culorilor se realizează prin conversia cadrelor video ce sunt capturate de cameră, convertind indicii de culoare în format HSV și se creează măști bazate pe game de culori predefinite. Se verifică pixelii care nu sunt zero din măști și astfel se detectează dacă acea culoare se află în captura video sau nu. După recunoașterea uneia dintre culorile prestabilite se efectuează o întârziere înainte de resetarea indicilor pentru a nu suprapune ciclurile de detectare.

Primele intervenții necesare pentru a ne asigura de funcționarea codului sunt bibliotecile Python ce ajută la procesarea imaginilor și a capturilor video. În Secvența 1 se pot vedea cele două importuri din Python.

Secvența 1 - Importuri captare și procesare video

```
import cv2  
import numpy as np
```

cv2 este un modul Python din librăria OpenCv, acesta oferă o interfață simplă și intuitivă pentru a accesa funcționalitățile OpenCv. Cu ajutorul cv2 se poate face citirea și scrierea imaginilor sau a videoclipurilor, se pot aplica filtre și se pot realiza transformări. De asemenea cu ajutorul cv2 putem detecta obiecte sau să analizăm imaginile.

NumPy (np) este o bibliotecă pentru computare numerică în Python. Poate ajuta la performarea operațiilor matematice și manipularea seturilor mari de date. Se folosește pentru diferite scopuri științifice. În cadrul acestei lucrări este necesar și folosit pentru captarea imaginilor, conversia în HSV din BGR, aplicarea măștii pentru detecția culorii.

În continuare, în Secvența 2 folosind cv2 se inițializează captura video.

Secvența 2 - Inițializare captură video

```
webcam_video = cv2.VideoCapture(0)
```

Funcția `cv2.VideoCapture()` face parte din biblioteca OpenCv. Aceasta preia argumentul numeric, ce reprezintă indexul dispozitivului de cameră care este utilizat. În cazul sistemului acesta, 0 este transmis pentru a reprezenta camera conectată. În fișierul Python, variabila `webcam_video` va fi folosită pentru a citi cadrele de la cameră. Se va folosi într-o secvență ce se repetă la infinit (`while True`) până când utilizatorul va alege oprirea programului.

Cum am menționat, se realizează conversia culorii din BGR în HSV, însă acest lucru se realizează în cadrul secvenței infinite. Înainte de a intra în aceasta, se declară variabilele pentru valorile maxime și minime ale fiecărei culori. În fișierul pentru detecția culorilor roșu și verde putem analiza acești indici în Secvența 3, iar pentru detecția culorilor galben și albastru în Secvența 4. Acestea sunt asemănătoare, însă bineînțeles diferă de la culoare la culoare.

Secvența 3 - Interval HSV roșu și verde

```
lower_red = np.array([0, 70, 50])  
upper_red = np.array([10, 255, 255])  
  
lower_green = np.array([90, 50, 50])  
upper_green = np.array([145, 255, 255])
```

Pentru detecția culorii roșu este folosit intervalul 0-10 pentru nuanță (hue), iar pentru verde 90-145. Indicii de saturație și luminozitate sunt aleși astfel pentru a oferi o aproximare cât mai clară a culorilor. Funcția `np.array()` este din biblioteca NumPy care preia parametrii și îi transformă într-o listă NumPy. În acest caz variabilele `lower_red` și `upper_red` respectiv `lower_green` și `upper_green` vor reține limitele HSV ale fiecărei culori.

Secvența 4 - Interval HSV galben și albastru

```
lower_blue = np.array([90, 50, 50])  
upper_blue = np.array([130, 255, 255])  
  
lower_yellow = np.array([20, 100, 100])  
upper_yellow = np.array([40, 255, 255])
```


Pentru detecția culorii albastru este folosit intervalul 90-130 pentru nuanță (hue), iar pentru galben 20-40. Indicii de saturație și luminozitate sunt aleși astfel pentru a oferi o aproximare cât mai clară a culorilor. Această parte se află în fișierul pentru detecția culorilor galben și albastru.

Pentru explicarea codului voi folosi fișierul pentru detecția galben și albastru, iar unde este necesar voi menționa ce este diferit în abordarea pentru roșu și verde, cum am făcut anterior. Majoritatea variabilelor folosite pentru detecție în cele două fișiere Python au aceeași structură, diferența fiind denumirea culorii, cum a fost prezentat, de exemplu, anterior `lower_green` și `lower_yellow`.

Recunoașterea culorilor se face pe rând, adică nu va putea detecta și galben și albastru în același timp, acest lucru se întâmplă deoarece este nevoie de acțiuni diferite pentru culori. Astfel pentru a ne asigura de această funcționalitate vom folosi două variabile de tip boolean `is_yellow_detected` și `is_blue_detected` care la începutul rulării au valoarea `false` (fals) și își schimbă această valoare în `true` (adevărat) în momentul în care camera detectează una dintre culori. Astfel dacă se detectează culoarea galben, `is_yellow_detected` își va schimba valoarea în adevărat. În Secvența 5 putem observa cum sunt declarate cele două variabile, înainte de începerea buclei infinite `while True`.

Secvența 5 - Declararea variabilelor ce indică recunoașterea sau nu a unei culori

```
is_yellow_detected = False  
is_blue_detected = False
```

Similar, în fișierul Python pentru detecția culorilor roșu și verde, vom găsi declararea variabilelor `is_green_detected` și `is_red_detected` făcută, adică fiindu-le atribuită valoarea `false`.

După importarea bibliotecilor, setarea de variabile și asigurarea că toți parametrii necesari sunt prezenți, vom intra în structura infinită `while True`. În Secvența 6 este prezentată bucata de cod ce ajută la capturarea frame-urilor video dar și conversia culorii din format BGR în HSV.

Secvența 6 - Capturarea frame-urilor video, conversie din BGR în HSV

```
success, video = webcam_video.read()  
img = cv2.cvtColor(video, cv2.COLOR_BGR2HSV)
```

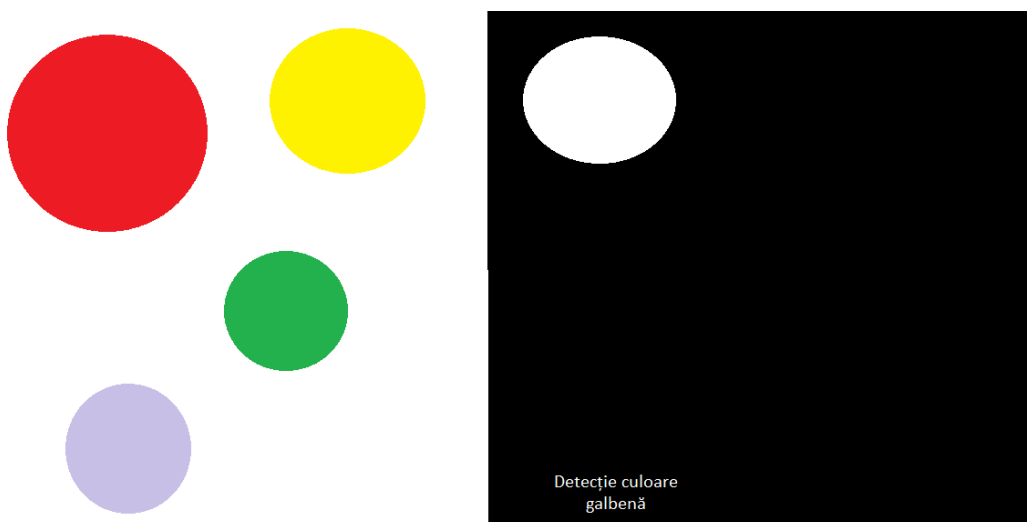
Funcția `webcam_video.read()` este folosită pentru capturarea frame-urilor video. Aceasta returnează valorile "success" și "video". Success este o variabilă de tip boolean

ce indică dacă citirea a fost sau nu realizată cu succes. Video este frame-ul capturat din video.

Funcția `cv2.cvtColor()` convertește frame-ul captat de la setările prestabilite de culoarea BGR (blue, green, red - albastru, verde, roșu) la HSV. Această funcție are doi parametri, intrarea "video" și codul de conversie al culorii `cv2.COLOR_BGR2HSV`. Rezultatul "img" va ajuta la recunoașterea culorilor din spectrul HSV. Această variabilă "img" va fi ulterior utilizată pentru limitarea culorilor și extragerea de informații și va ajuta la crearea măștilor specifice pentru fiecare culoare în parte.

Cum am menționat anterior, pentru determinarea culorilor, am ales varianta de a crea o mască binară. Aceste măști se creează individual pentru fiecare culoare. Aceste măști se fac pe variabila `img`, care este intrarea. Maska funcționează astfel încât culoarea căutată va fi formată din pixeli albi, iar orice altă culoare va avea pixeli negrii. Un astfel de exemplu se poate observa în Figura 11.

Figura 11 - Exemplu mască pentru recunoașterea culorilor



Ambele fișiere Python conțin crearea a câte două măști reprezentând detecția roșu și verde respectiv galben și albastru. În Secvența 7 se poate observa codul pentru detecția roșu și verde iar în Secvența 8 codul pentru detecția galben și albastru.

Secvența 7 - Crearea de mască pentru detecția roșu și verde

```
mask_red = cv2.inRange(img, lower_red, upper_red)
mask_green = cv2.inRange(img, lower_green, upper_green)
```

Secvența 8 - Crearea de mască pentru detecția galben și albastru

```
mask_blue = cv2.inRange(img, lower_blue, upper_blue)
mask_yellow = cv2.inRange(img, lower_yellow, upper_yellow)
```

Funcția `cv2.inRange()` îndeplinește principalul scop, acela de a crea masca binară aplicată pe variabila `img`. “`img`” este așa cum am menționat anterior un șir NumPy care reprezintă de fapt imaginea din captura video a camerei. `lower_CULOARE` și `upper_CULOARE` sunt pixelii ce vor fi căutați în `img`, orice se găsește în acest interval va fi setat ca alb în mască, în timp ce oricare altă nuanță va avea pixelii negrii. Rezultatul este variabila `mask_CULOARE` și este o mască binară, având aceleași dimensiuni ca `img`.

Pentru a verifica apariția unei culori în captura video, vom avea pentru fiecare culoare câte o structură de tipul `if` (dacă), în care se vor face diferite verificări și acțiuni specifice. Codul este similar pentru toate culorile, dar pentru exemplificare vom alege discuta despre culoarea albastru. În Secvența 9 putem observa condiția necesară pentru a decide dacă o culoare este detectată sau nu.

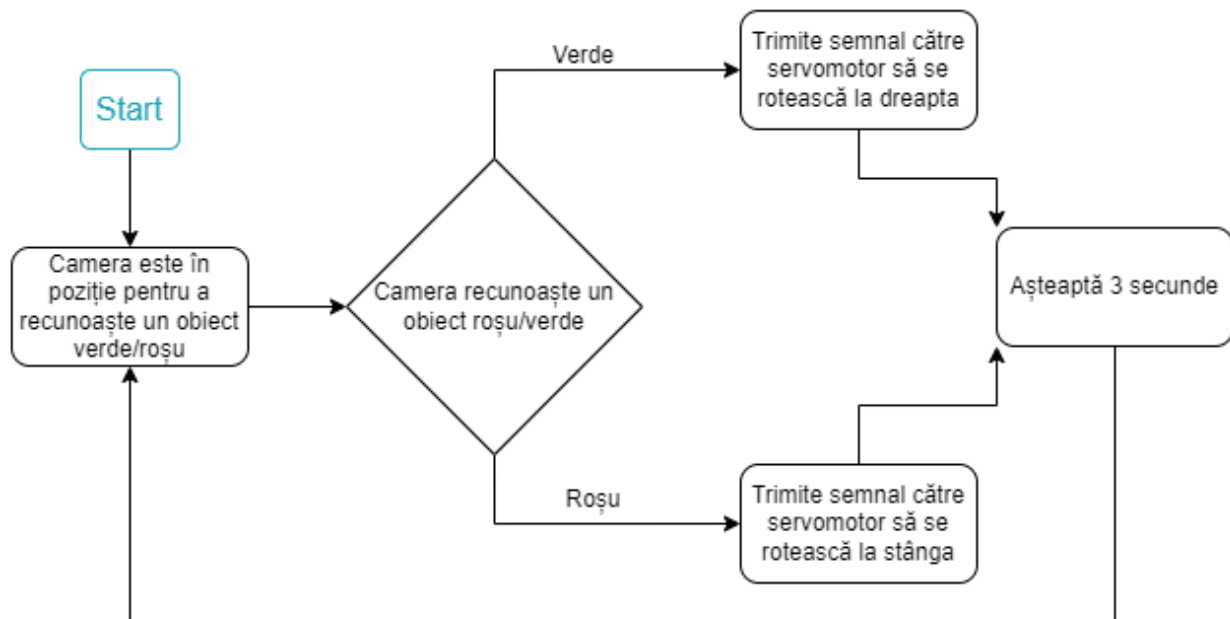
Secvența 9 - Condiție receptare culoare (albastru)

```
if cv2.countNonZero(mask_blue) > 0 and not is_blue_detected and not is_yellow_detected:
    is_blue_detected = True
```

`cv2.countNonZero()` este o funcție din biblioteca OpenCv care numără câți pixeli non-zero (culoarea alb) sunt prezenți în `mask_blue`. Deci verifică dacă există un obiect de acea culoare prin verificarea dacă masca creată anterior a creat pixeli albi. De asemenea, concomitent, pentru a evita o reacție continuă de la sistem, verificăm dacă este detectată deja vreo culoare (dintre cele pe care le analizăm noi), în cazul sistemului pentru recunoașterea galben și albastru ne dorim ca acestea să nu fie prezente, iar în cazul roșu și verde ne dorim ca acestea să fie analizate și să nu fie prezente. În cazul în care aceste trei condiții simultane sunt îndeplinite, variabila `is_CULOARE_detected` își va schimba valoarea de adevăr, astfel va deveni `True` (adevărat), adică este receptată culoarea.

Schema unde se poate analiza logica din spatele algoritmului de detecție a culorii se găsește în Figura 12, aici este prezentat parcursul din momentul pornirii sistemului până la intervenția servomotorului, deci practic intervenția camerei și cum funcționează algoritmul de detecție al culorii. Am folosit ca exemplu detecția de roșu și verde, logica pentru galben și albastru funcționând identic.

Figura 12 - Schemă algoritm detectare culoare



Pentru a ne asigura că servomotorul nu primește semnal constant de la Raspberry Pi cum că ar fi detectat una dintre culori, vom dori ca verificarea dacă o culoare deja există să se întâmple o dată la 3 secunde. Acest lucru se poate observa în Secvența 10.

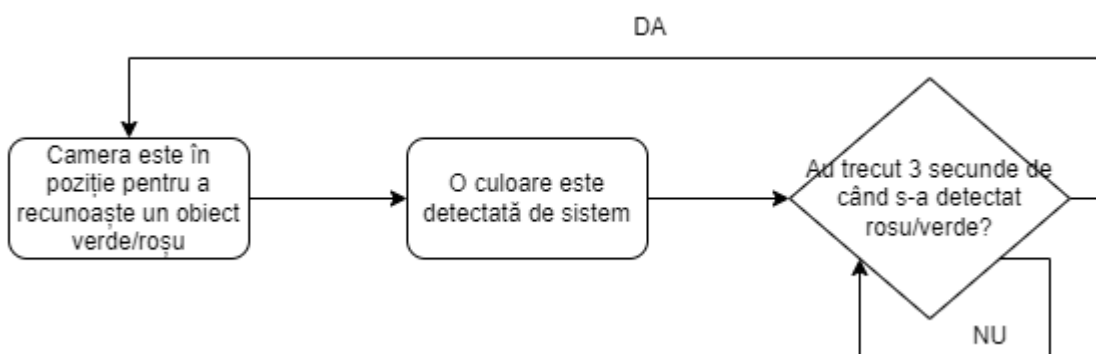
Secvența 10 - Import bibliotecă pentru timp și secvență așteptare 3 secunde

```
import time
.
.
.
if cv2.countNonZero(mask_blue) > 0 and not is_blue_detected and not is_yellow_detected:
    is_blue_detected = True
    last_detection_time = time.time()
.
.
.
if cv2.countNonZero(mask_yellow) > 0 and not is_yellow_detected and not is_blue_detected:
    is_yellow_detected = True
    last_detection_time = time.time()
.
.
.
if time.time() - last_detection_time > 3:
    is_yellow_detected = False
    is_blue_detected = False
.
.
```

“time” este un modul Python care oferă diferite funcții pentru operațiile cu timp. Variabila `last_detection_time` salvează timpul actual în care s-a detectat o culoare prin apelarea funcției `time.time()` și se va folosi pentru a reține ultimul moment în care a fost recunoscută o culoare. Timpul curent este salvat în secunde. Spre finalul fișierului se va găsi condiția `time.time() - last_detection_time > 3`, unde practic se face diferența dintre timpul curent și timpul de la ultima detectare a unei culori. Dacă acest timp este mai mare de 3 secunde, cele două variabile `is_CULOARE_detected` vor primi valoarea fals, deci se poate intra iar în structura condițională `if` pentru a verifica dacă o culoare este identificată.

Schema unde se exemplifică logica tocmai prezentată se găsește în Figura 13.

Figura 13 - Schemă funcționare timp de așteptare



În continuare semnalele vor fi transmise la servomotor, care va efectua mișcări specifice (stânga sau dreapta) pentru fiecare dintre culori.

5.2. Programarea servomotor

Pentru a putea lucra cu servomotorul, este necesar să importăm bibliotecile și să configurăm GPIO. În Secvența 11 se poate observa “`import RPi.GPIO as GPIO`” care importă biblioteca `RPi.GPIO`. Aceasta este un modul Python pentru controlarea pinilor GPIO de pe plăcuța Raspberry Pi. `GPIO.setmode(GPIO.BOARD)` face legătura dintre pinii GPIO și pinii fizici de pe plăcuță, am menționat anterior în lucrare faptul că numerotarea este diferită, iar pentru a ne asigura de corectitudinea conectării firelor prin “`setmode`” vom trimite referința la numerotarea fizică de pe Raspberry Pi, adică `GPIO.BOARD`. În continuare vom face setup-ul prin `GPIO.setup()`, configurând pinul 11 ca ieșire, deci prin acesta se vor trimite semnalele.

Secvența 11 - Import bibliotecă și setup GPIO

```
import RPi.GPIO as GPIO
.
.
.
GPIO.setmode(GPIO.BOARD)
.
.
.
GPIO.setup(11, GPIO.OUT)
servo1 = GPIO.PWM(11, 50)
.
.
.
servo1.start(0)
.
.
```

“servo1 = GPIO.PWM(11, 50)” crează un nou obiect PWM prin variabila servo1. Aceasta este inițializată prin pinul 11, având o frecvență de 50 Hz (Hertz). “servo1.start(0)” pornește servo1 cu un ciclu inițial 0, adică stare de repaus inițială.

După ce setup-ul este pregătit, intrând în structura infinită while, apoi în structura condițională discutată anterior în care verificăm dacă o culoare este detectată, în cazul unei detecții servomotorul se va mișca stânga sau dreapta, în funcție de culoarea detectată. În Secvența 12 se poate observa codul servomotorului pentru mișcare la dreapta, iar în Secvența 13 se poate observa codul servomotorului pentru mișcare la stânga.

Secvența 12 - Rotire servomotor la dreapta și revenire la poziția inițială

```
servo1.ChangeDutyCycle(2.5)
time.sleep(2)
servo1.ChangeDutyCycle(5)
time.sleep(0.5)
servo1.ChangeDutyCycle(0)
```

Secvența 13 - Rotire servomotor la stânga și revenire la poziția inițială

```
servo1.ChangeDutyCycle(7.5)  
time.sleep(2)  
servo1.ChangeDutyCycle(5)  
time.sleep(0.5)  
servo1.ChangeDutyCycle(0)
```

Vom analiza concomitent cele două părți. “servo1.ChangeDutyCycle(2.5)” corespunde mișcării la dreapta, setând ciclul la 2.5%, iar “servo1.ChangeDutyCycle(7.5)” are celeași caracteristici, setând ciclul la 7.5%, rotind servomotorul la stânga. “time.sleep(2)” va opri motorul pentru două secunde, astfel se va opri în poziția curentă, obiectele vor avea timpul necesar să treacă prin partea corespunzătoare sortării. “servo1.ChangeDutyCycle(5)” va întoarce servomotorul în direcția opusă, iar după 0.5 secunde “servo1.ChangeDutyCycle(0)” va readuce servomotorul în poziția inițială.

Finalul fișierelor Python constă în ieșirea din program, închiderea camerei și închiderea ferestrelor OpenCv care au fost deschise pe parcursul rulării. În Secvența 14 se pot găsi instrucțiunile necesare realizării acestor pași.

Secvența 14 - Comenzi terminare program

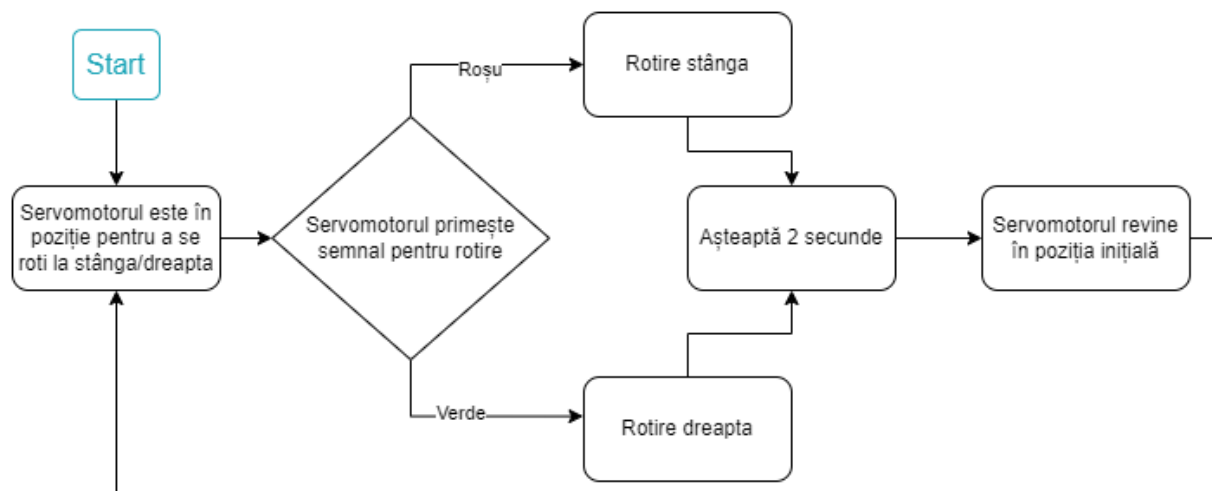
```
key = cv2.waitKey(1)  
if key == 27:  
    break  
  
webcam_video.release()  
  
cv2.destroyAllWindows()
```

Primele trei linii din Secvența 14 se află în interiorul structurii infinite while True. Tasta Escape corespunde cu valoarea 27, astfel dacă utilizatorul apasă Escape în timpul rulării, programul se va opri. “webcam_video.release()” eliberează sursa video a camerei. Este necesară deoarece pot apărea probleme la următoarea rulare în cazul în care camera nu se închide la finalul fiecărui program. “cv2.destroyAllWindows()” închide toate ferestrele OpenCv ce s-au deschis în timpul execuției. Asemănător este pentru a ne asigura că programul se termină corect și cu succes.

Pentru a exemplifica logica prin care funcționează servomotorul, în Figura 14 se poate observa ciclul pe care îl urmează acesta de când este pornit întreg procesul. Pentru

exemplificare am folosit culorile roșu și verde, iar pentru galben și albastru funcționează identic.

Figura 14 - Schemă logică funcționare servomotor



Astfel partea de recunoaștere a culorii și controlul servomotorului sunt realizate.

5.3. Dezvoltarea aplicației Android Studio

Despre aplicația Android voi începe prin a prezenta fișierul XML. Acesta ajută și are scopul de a defini interfața cu utilizatorul, fiind folosit un ConstraintLayout cu ajutorul căruia poziționăm butoanele și elementele, oferindu-le diferite dimensiuni. În interiorul acestuia sunt mai multe LinearLayouts care au scopul de a aranja orizontal și vertical elementele. Primul LinearLayout (button_layout_1) conține două butoane "Start Application Green-Red" și "Stop Application Green-Red", cel de-al doilea (button_layout_2) conține următoarele seturi de butoane "Start Application Blue-Yellow" și "Stop Application Blue-Yellow". Ultimul LinearLayout este special pentru butonul "Shutdown". Toate aceste Layout-uri conțin specificații de lățime, înălțime, margini, text, culoarea textului, nuanța de fundal și constrângerile de poziționare. Acest cod se poate urmări în Secvența 15 unde este prezentat ConstraintLayout-ul, în interiorul acestuia se află elementele prezentate anterior.

Secvența 15 - XML aplicație Android

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    //LinearLayout1 ce conține două butoane "Start Application Green-Red" și "Stop Application Green-Red"
    //LinearLayout2 ce conține două butoane "Start Application Blue-Yellow" și "Stop Application Blue-Yellow"
    //LinearLayout3 ce conține butonul "Shutdown"

</androidx.constraintlayout.widget.ConstraintLayout>
```

Prin “<?xml version=“1.0” encoding=“utf-8”?” se specifică versiunea XML pe care o folosesc, adică 1.0, versiunea de codificare folosită este menționată tot aici. Această declarație fiind necesară pentru a informa XML-ul, asigurându-ne astfel că documentul este procesat corect de către software-ul care îl citește.

Codul pentru realizarea propriu-zisă a aplicației Android se găsește în MainActivity.java. Una dintre principalele componente este metoda “onCreate” care este apelată atunci când se creează o activitate. În Secvența 16 “setContentView(R.layout.activity_main);” setează fișierul XML ca și conținut de vizualizare pentru MainActivity.java, practic definește interfața cu utilizatorul. În continuare se inițializează butoanele și sunt atribuite variabilele folosind metoda “findViewById”, asociind fiecare buton vizibil utilizatorului cu o acțiune. Acest lucru se realizează printr-un listener, astfel când unul dintre butoane este apăsat de utilizator, codul din interiorul metodei se va executa. Vom alege pentru exemplificare primul buton prezent în metodă. “startAppGRButton.setOnClickListener” setează listener-ul pentru butonul “startAppGRButton”, se specifică acțiunea ce trebuie realizată în momentul în care butonul este apăsat. “public void onClick(View v)” definește acțiunile care trebuie efectuate atunci când se face click pe buton. Parametrul “v” reprezintă butonul în sine în acest caz.

Secvența 16 - Metoda onCreate() și conținut

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    Button startAppGRButton = findViewById(R.id.start_application_gr_button);
    Button startAppBYButton = findViewById(R.id.start_application_by_button);
    Button stopAppGRButton = findViewById(R.id.stop_application_gr_button);
    Button stopAppBYButton = findViewById(R.id.stop_application_by_button);
    Button shutdownButton = findViewById(R.id.shutdown_button);

    startAppGRButton.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View v) {
            executeCommand(commandToStartAppGR);
        }
    });

    // similar pentru butoanele startAppBYButton, stopAppGRButton, stopAppBYButton, shutdownButton
}
```

“@Override” înainte de metoda onCreate() este folosit pentru a indica faptul că această metodă din clasa MainActivity are scopul de a suprascrie aceeași metodă definită în superclasă (AppCompatActivity). În cazul scrierii înainte de metoda onClick() este folosit pentru a indica faptul că această metodă suprascrie aceeași metodă definită în interfața View.OnClickListener.

La final este apelată metoda onResume() pentru a permite controlul atunci când o activitate intră în prim-plan și devine din nou vizibilă pentru utilizator. Astfel această metodă este esențială pentru gestionarea interfeței cu utilizatorul.

5.4. Comunicare aplicație - Raspberry Pi

Comunicarea dintre aplicația Android și Raspberry Pi se realizează prin SSH, astfel se obține acces securizat. Pentru a configura această conexiune avem nevoie de mai multe date specifice, date preluate de la Raspberry Pi. Astfel în Secvența 17 se poate observa inițializarea variabilelor cu datele de conectare.

Secvența 17 - Inițializarea variabilelor și datele de conectare prin SSH

```
private String raspberryPiIp = "192.168.124.2";  
private String sshUsername = "licenta";  
private String sshPassword = "parola";  
private String commandToStartAppGR = "python GR.py";  
private String commandToStartAppBY = "python BY.py";  
private String commandToStopAppGR = "pkill -f GR.py";  
private String commandToStopAppBY = "pkill -f BY.py";  
private String commandToShutdown = "sudo /sbin/shutdown now";
```

IP-ul este căutat și preluat din plăcuță, de asemenea username-ul plăcuței folosite de mine este "licenta", iar parola specifică a acestuia este "parola". În continuare `private commandToStartAppCULORI = "python CULORI.py"` va executa în terminalul Raspberry Pi comenzile specifice pentru rularea fișierelor Python menționate. Comenzile de oprire ale programelor folosesc comanda "pkill" cu opțiunea "-f". Această comandă este specifică Linux, astfel alegând opțiunea "-f" se va căuta programul ce rulează menționat și se va trimite semnalul prin care aceasta se va opri. Ultima comandă, cea de shutdown inițiază oprirea întregului sistem al Raspberry Pi la momentul rulării comenzii în terminal.

Pentru trimiterea propriu-zisă a acestor semnale folosim pachetele JSch [23], aceasta este o bibliotecă Java ce ajută implementarea SSH, de ajutor în această lucrare pentru transferul de informație. În Secvența 18 se prezintă implementarea metodei "executeCommand()" prin care se realizează comunicarea.

Secvența 18 - Metoda prin care se realizează comunicarea SSH

```
private void executeCommand(final String command) {  
    new AsyncTask<Void, Void, Void>() {  
        @Override  
        protected Void doInBackground(Void... params) {  
            try {  
                JSch jsch = new JSch();  
                Session session = jsch.getSession(sshUsername, raspberryPiIp, 22);  
                session.setPassword(sshPassword);  
                session.setConfig("StrictHostKeyChecking", "no");  
                session.connect();  
  
                ChannelExec channel = (ChannelExec) session.openChannel("exec");  
                channel.setCommand(command);  
                channel.connect();  
                channel.disconnect();  
  
                session.disconnect();  
            } catch (JSchException e) {  
                e.printStackTrace();  
            }  
            return null;  
        }  
    }.execute();  
}
```

Cum am menționat, folosim biblioteca JSch, iar în Secvența 18 se poate observa cum se creează o nouă instanță și se configurează o sesiune prin metoda „getSession()” cu numele de utilizator SSH, adresa IP Raspberry Pi și portul 22. Prin dezactivarea „StrictHostKeyChecking” se evită verificarea cheii serverului în timpul conexiunii SSH. După această parte în care se stabilește conexiunea, se deschide un canal „exec” pentru a executa o comandă pe serverul de la distanță. În final sesiunea este deconectată, terminând conexiunea SSH.

6. TESTARE

În acest capitol voi prezenta reacția sistemului la diferite situații care ar putea face recunoașterea culorii sau sortarea îngreunată.

Pentru a testa funcționalitatea sistemului în timp voi verifica câte bile colorate poate sorta sistemul corect dintre 25 de bile. Știm că între fiecare moment în care sistemul caută o culoare nouă dintre cele pe care le poate recunoaște este un timp de așteptare de 3 secunde.

Primul scenariu este acela în care bilele sunt introduse pe rând, așteptând cele 3 secunde înainte de a introduce o nouă bilă colorată. Pentru acest test vom folosi sortarea galben-albastru. Rezultatele testului sunt prezente în Tabelul 4. Dacă așteptăm până servomotorul revine la poziția inițială, toate bilele sunt sortate corect.

Tabelul 4 - Rezultate test 1

Număr bile sortate corect	Număr bile sortate greșit
25	0

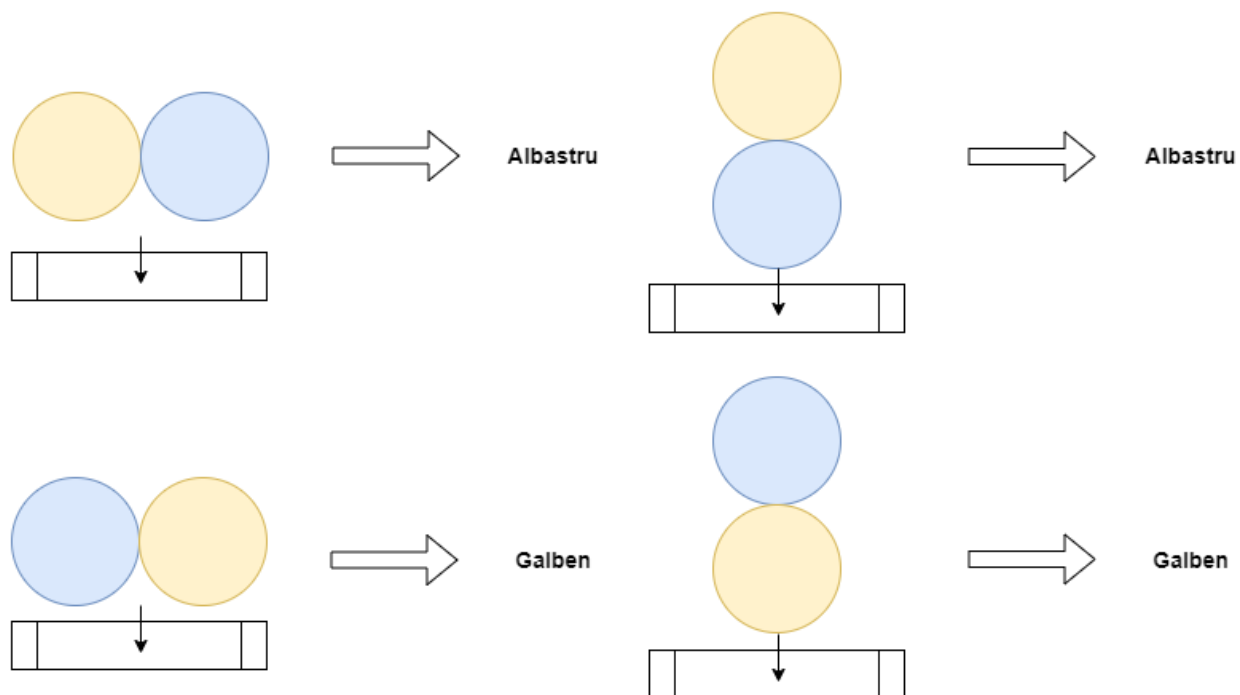
Cel de-al doilea scenariu ar fi acela în care bilele sunt introduse mult mai repede, practic analizăm reacția servomotorului despre care știm că și acesta are delay-uri de 2 și 0.5 secunde. Pentru acest test vom folosi sortarea galben-albastru, rezultatele se pot observa în Tabelul 5. Introducând bilele mult mai repede, putem observa că există erori datorate faptului că timpul de 3 secunde dintre o recunoaștere și alta nu este îndeplinit înainte de a introduce o nouă bilă.

Tabelul 5 - Rezultate test 2

Număr bile sortate corect	Număr bile sortate greșit
17	8

În continuare, vom testa reacția sistemului la două bile de culori diferite introduse simultan. Se va analiza reacția camerei dar și a servomotorului. Pentru acest test vom avea 4 variante de a introduce bilele, reprezentate în Figura 15. Aceleași rezultate sunt prezente după testarea de 5 ori prin același model. Am introdus bilele albastre și galbene în 4 variante, vertical cu galben prima bilă, rezultatul fiind galben și vertical cu albastru prima bilă, rezultatul fiind albastru. Când am introdus bilele orizontal rezultatul a fost culoarea din dreapta, practic prima captată de cameră.

Figura 15 - Rezultate test 3



7. CONCLUZII ȘI PERSPECTIVE

Acest sistem oferă o soluție eficientă de sortare a obiectelor bazându-se pe culorile acestora. Este conceput pentru persoanele daltoniste, reușind să diferențieze roșu de verde și galben de albastru. Este un sistem conceput în maniera IoT, este ușor de utilizat prin aplicația ce oferă o interfață utilizatorilor, se poate transporta dacă este necesar și poate fi folosit cu ușurință folosind o baterie externă, neavând constrângeri de a fi nevoit să ai o priză sau o sursă de energie la îndemână.

Mi-am dorit de asemenea ca proiectul meu să ajute la conștientizarea acestor probleme de vedere. Consider că indiferent de afecțiunile pe care le avem, cu toții ar trebui să putem realiza acțiuni simple, iar acest proiect consider că ajută la determinarea culorilor cu succes de către persoanele daltoniste.

În acest moment proiectul funcționează în două părți, sortare roșu și verde respectiv galben și albastru. Pe viitor cred că ar fi de folos să poată sorta toate cele patru culori în același timp. Pentru a realiza această funcționalitate aș mai conecta două servomotoare, ce ar fi amplasate fizic în continuarea celui deja prezent. Codul Python l-aș rescrie într-un singur fișier, și ar funcționa destul de asemănător ca proiectul de acum, cu câteva adăugiri. Primul servomotor ar sorta, de exemplu, la dreapta roșu și verde și la stânga galben și albastru. Ajungând la servomotorul 2 sau 3, acestea ar avea comportament similar ca și primul. Presupunem că avem un obiect roșu, servomotorul 1 ar sorta obiectul la dreapta, de unde ar ajunge la servomotorul 2, aici sistemul poate iar sorta, după același principiu, stânga și dreapta. Iar la final ar fi patru containere pentru toate culorile ce se pot sorta.

O altă funcționalitate care cred că ar ajuta mult acest sistem ar fi o aplicație mobilă mai complexă, de exemplu în care să apară și statistici. Acest lucru s-ar putea realiza prin implementarea unui server. Cu ajutorul solicitărilor HTTP (protocol trimitere date) aș putea prelua și adăuga date într-o bază de date.

Cu aceste două funcționalități suplimentare consider că acest sistem ar fi alegerea perfectă dacă dorești să sortezi obiecte în funcție de culoarea lor.

Ceea ce consider că a fost o provocare în realizarea acestui proiect a fost funcționalitatea de recunoaștere în mod corect al culorii. Am menționat în lucrare că se folosesc limite HSV, iar pentru culoarea verde a fost foarte greu să găsesc limitele potrivite deoarece camera recunoștea o gamă foarte mare de nuanțe, închise sau deschise, ca fiind verde. De asemenea, sincronizarea camerei cu servomotorul a fost mai grea decât m-am așteptat, a fost nevoie să adaug modul de timp și așteptarea pentru a nu se suprapune comenzile și acțiunile lor.

Funcționalitățile ce se regăsesc în CSort consider că oferă un sprijin persoanelor daltoniste și pot spune că acest proiect este un prim pas spre conștientizare, de asemenea spre dezvoltarea sistemelor de ajutor persoanelor cu afecțiuni, în acest caz afecțiuni de vedere.

8. REFERINȚE BIBLIOGRAFICE

- [1] *** <https://www.covisn.com/interesting-facts-about-how-many-people-are-color-blind/>
accesare iunie 2023
- [2] *** <https://www.zoomax.com/low-vision-information/What-is-Color-Blindness-.html>
accesare iunie 2023
- [3] ***
<https://play.google.com/store/apps/details?id=com.colorblindpal.colorblindpal&hl=en&gl=US&pli=1> accesare iunie 2023
- [4] ***
<https://steinertglobal.com/magnets-sensor-sorting-units/sensor-sorting/colour-sorting-system/steinert-kss-cl/> accesare iunie 2023
- [5] - F. Bianconi, A. Fernández, F. Smeraldi, G. Pascoletti, "Colour and Texture Descriptors for Visual Recognition: A Historical Overview", *MDPI - Publisher of Open Access Journals*, 19 noiembrie 2021.
- [6] - A. Shamsan, E. M. Senan, Hamzeh S.h A. Shatnawi, "Automatic Classification of Colour Fundus Images for Prediction Eye Disease Types Based on Hybrid Features", *MDPI - Publisher of Open Access Journals*, 11 mai 2023.
- [7] - A. C. Abad, D. D. Ligutan, E. P. Dadios, L. J. S. Cruz, M. C. D.P. Del Rosario, J. N. S. K.I "Fuzzy Logic-Controlled 6-DOF Robotic Arm Color-based Sorter with Machine Vision Feedback", *International Journal of Advanced Computer Science and Applications* (IJACSA), Vol. 9, No. 5, 2018
- [8] - R. A. Magadum, U. L. Bombale, "Intelligent Color Based Object Sorting Using Robotics", *Journal of Robotics Research and Development* (IJRRD), iunie 2019
- [9] - L. Fina, T. Mascarenhas, C. Smith, H.i E. Sevil, "Object Detection Accuracy Enhancement in Color based Dynamic Sorting using Robotic Arm", *34th Florida Conference on Recent Advances in Robotics*, mai 13-14, 2021
- [10] *** <https://www.spiceworks.com/tech/networking/articles/what-is-raspberry-pi/>
accesare iunie 2023
- [11] *** <http://www.raspbian.org/> accesare iunie 2023
- [12] *** <https://www.linux.com/what-is-linux/> accesare iunie 2023
- [13] ***
<https://www.coursera.org/articles/what-is-python-used-for-a-beginners-guide-to-using-python> accesare iunie 2023
- [14] *** <https://raspberrypi.org/install-opencv-on-raspberry-pi/> accesare iunie 2023
- [15] *** <https://developer.android.com/studio/intro> accesare iunie 2023
- [16] *** https://www.java.com/en/download/help/whatis_java.html accesare iunie 2023
- [17] *** <https://projects.raspberrypi.org/en/projects/getting-started-with-picamera/1>
accesare iunie 2023
- [18] *** <https://www.arducam.com/raspberry-pi-camera-pinout/> accesare iunie 2023

- [19] *** https://en.wikipedia.org/wiki/HSL_and_HSV accesare iunie 2023
- [20] *** <https://datasheetpdf.com/pdf/791970/TowerPro/SG90/1> accesare iunie 2023
- [21] *** <https://pinout.xyz/pinout/wiringpi> accesare iunie 2023
- [22] *** <https://www.techtarget.com/searchsecurity/definition/Secure-Shell> accesare iunie 2023
- [23] *** <http://www.jcraft.com/jsch/> accesare iunie 2023
- [24] *** <https://chat.openai.com/> accesare iunie 2023

ANEXA 1: Ghidul de utilizare al sistemului

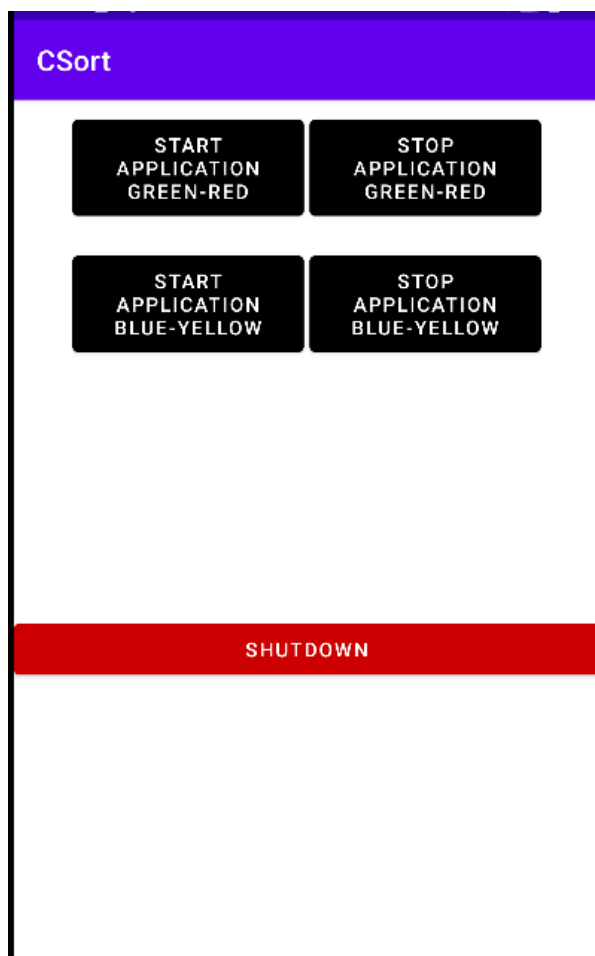
În această parte voi prezenta cum se folosește sistemul, pornire Raspberry Pi, utilizarea aplicației Android și cum se amplasează obiectele pentru sortare.

Sistemul are deja conectate camera și servomotorul, iar pentru a porni Raspberry Pi tot ce trebuie făcut este să conectăm la o sursă de curent. Se poate folosi cu o baterie externă, pentru a nu depinde de priză sau se poate conecta cu ajutorul sursei de alimentare.

Pentru ca aplicația Android și Raspberry Pi să comunice este necesar să fie conectate la aceeași rețea.

Aplicația se poate deschide de pe telefon și în funcție de culorile pe care vrem să le sortăm putem da click pe “START APPLICATION GREEN-RED” sau “START APPLICATION BLUE-YELLOW”. În Figura 16 se poate observa ecranul cu butoanele sale.

Figura 15 - Ecran aplicație Android



În funcție de ce sortare am ales să facem, pentru a porni una nouă, va trebui să apăsăm pe “STOP APPLICATION GREEN-RED” sau “STOP APPLICATION BLUE-YELLOW”.

Butonul “SHUTDOWN” va opri alimentarea plăcuței Raspberry Pi, astfel și conexiunea SSH se va opri deci aplicația nu va mai răspunde la comenzi.

Obiectele colorate ar trebui introduse de către utilizator cum este exemplificat în Figura 17. În chenarul 1 este poziția în care bila se poziționează. Acestea i se dă drumul și ea alunecă prin inerție până la chenarul 4, unde este aflat servomotorul. În chenarul 2 se află Raspberry Pi, în chenarul 3 se află camera iar în chenarul 5 este paleta ce este mișcată de servomotor stânga sau dreapta.

Figura 17 - Utilizare sistem fizic



**DECLARAȚIE DE AUTENTICITATE A
LUCRĂRII DE FINALIZARE A STUDIILOR***

Subsemnatul BICIUȘCĂ ADELA-MARIA

legitimată cu C.I. seria SL nr. 000941

CNP 6000705282204

autorul lucrării SISTEM DE SORTARE OBIECTE ÎN
FUNCȚIE DE CULOARE

elaborată în vederea susținerii examenului de finalizare a studiilor de
LICENȚĂ organizat de către Facultatea

AUTOMATICĂ ȘI CALCULATOARE din cadrul Universității
Politehnica Timișoara, sesiunea Iunie a anului universitar

2023, coordonator CONF. DR. ING. RĂZVAN BOGDAN, luând în
considerare art. 34 din *Regulamentul privind organizarea și desfășurarea examenelor de
licență/diplomă și disertație*, aprobat prin HS nr. 109/14.05.2020 și cunoscând faptul că în
cazul constatării ulterioare a unor declarații false, voi suporta sancțiunea administrativă
prevăzută de art. 146 din Legea nr. 1/2011 – legea educației naționale și anume anularea
diplomei de studii, declar pe proprie răspundere, că:

- această lucrare este rezultatul propriei activități intelectuale;
- lucrarea nu conține texte, date sau elemente de grafică din alte lucrări sau din alte surse fără ca acestea să nu fie citate, inclusiv situația în care sursa o reprezintă o altă lucrare/alte lucrări ale subsemnatului;
- sursele bibliografice au fost folosite cu respectarea legislației române și a convențiilor internaționale privind drepturile de autor;
- această lucrare nu a mai fost prezentată în fața unei alte comisii de examen/prezentată public/publicată de licență/diplomă/disertație;
- În elaborarea lucrării am utilizat instrumente specifice inteligenței artificiale (IA) și anume CHAT GPT (denumirea) OPENAI (sursa), pe care le-am citat în conținutul lucrării/nu am utilizat instrumente specifice inteligenței artificiale (IA)¹.

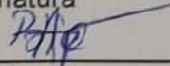
Declar că sunt de acord ca lucrarea să fie verificată prin orice modalitate legală pentru confirmarea originalității, consimțind inclusiv la introducerea conținutului său într-o bază de date în acest scop.

Timișoara,

Data

25.06.2023

Semnătura



¹*Declarația se completează de student, se semnează olograf de acesta și se inserează în lucrarea de finalizare a studiilor, la sfârșitul lucrării, ca parte integrantă.

Se va păstra una dintre variante: 1 - s-a utilizat IA și se menționează sursa 2 – nu s-a utilizat IA